



南京林业大学
NANJING FORESTRY UNIVERSITY

FeHALS 航路规划与激光仿真系统 API 技术文档

课程名称： “智能 +”GIS 设计与开发

专业： 测绘工程

组别： 7 组

年级： 2023 级

小组成员：	王孜悠	2310604116
	梅康凯	2310604109
	唐如意	2310604115
	杨杰瑞	2310604122
	温璞	2310604117
	刘继明	2110604112
	李英杰	2310604106
	杜仲宇	2310604103
	施祺	2310604113
	向宇鸣	2310604118

指导老师： 隋铭明 那嘉明 陈动 朱杰

批改日期：

提交日期： 2026 年 9 月 16 日

目录

- 1 系统概述 3
 - 1.1 项目背景 3
 - 1.2 系统架构 3
- 2 相关技术 3
 - 2.1 HELIOS++ 仿真框架 3
 - 2.2 Web 三维可视化与 Three.js 4
 - 2.3 Vue 3 前端框架 4
 - 2.4 FastAPI 与后端异步任务 4
- 3 系统需求分析与设计 4
 - 3.1 用户需求分析 4
 - 3.2 系统总体架构 4
 - 3.3 功能优先级划分 4
- 4 系统设计与实现 5
 - 4.1 系统模块划分 5
 - 4.2 三维场景渲染与航点交互 5
 - 4.3 前端组件与状态管理 5
 - 4.4 后端 API 接口设计 5
 - 4.5 航迹与配置文件生成 6
 - 4.6 HELIOS++ 调用与点云渲染 7
 - 4.7 项目组织结构 7
- 5 部署与运行指南 8
 - 5.1 环境要求 8
 - 5.2 安装步骤 8
 - 5.3 启动脚本 9
 - 5.4 配置说明 10
- 6 API 接口详解 11
 - 6.1 REST API 接口 11
 - 6.1.1 模型管理接口 11
 - 6.1.2 航迹管理接口 11
 - 6.1.3 仿真执行接口 12
 - 6.1.4 结果管理接口 12
 - 6.1.5 环境诊断与缓存管理 13
 - 6.2 WebSocket 接口 13
 - 6.2.1 日志推送 13

6.3	数据模型定义	13
7	前端实现详解	15
7.1	Vue 3 应用架构	15
7.1.1	组件结构	15
7.1.2	3D 场景组件实现	15
7.2	Three.js 集成与 3D 交互	20
7.2.1	场景管理组合式函数	20
7.2.2	航点交互系统	35
8	后端实现详解	36
8.1	FastAPI 应用架构	36
8.1.1	主应用配置	36
8.1.2	配置管理系统	36
8.2	API 路由实现	37
8.2.1	REST 路由	37
8.3	业务服务层	44
8.3.1	航迹生成服务	44
8.3.2	点云处理服务	46
A	部署与配置管理	51
A.1	环境变量配置	51
A.2	Docker 部署方案	51
A.3	服务监控与日志	52
B	开发规范与最佳实践	52
B.1	代码规范	52
B.2	API 设计原则	52
C	故障排查指南	52

1. 系统概述

1.1. 项目背景

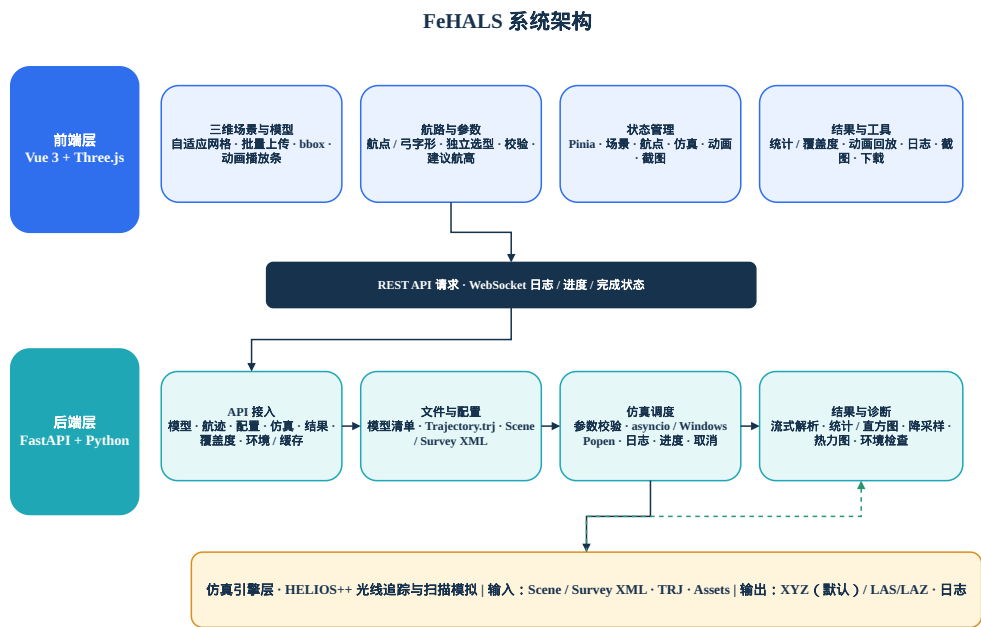
FeHALS (Frontend of HELIOS++ for Airborne Laser Scanning) 是一个基于 Web 的三维可视化航路规划与激光仿真系统，旨在解决 HELIOS++ 原生命令行交互复杂、缺乏三维可视化航路规划能力的问题。本系统为航测工程师、遥感研究人员以及相关专业师生提供一个直观易用的平台，支持从场景构建、航路设计、仿真执行到结果可视化的全流程操作。

1.2. 系统架构

系统采用浏览器/服务器 (B/S) 三层架构：

- 前端层：基于 Vue 3 与 Three.js 构建，负责三维场景渲染、航点交互编辑、参数配置与结果展示
- 后端层：基于 FastAPI 实现，负责 API 服务、文件生成、仿真调度与数据处理
- 仿真引擎层：HELIOS++ 激光扫描仿真引擎，独立运行并由后端调用

Fig. 1: FeHALS 系统架构与数据流



2. 相关技术

2.1. HELIOS++ 仿真框架

HELIOS++ 是一款基于 C++ 的开源激光雷达仿真框架，采用光线追踪技术对激光脉冲在三维场景中的传播过程进行建模，能够模拟机载、地面、车载与无人机等多种平台，支持全波形 (full-waveform) 输出与 LAS、LAZ、XYZ 等多种点云格式。其仿真配置以 XML 文件组织：场景 (scene) 文件定义参与扫描的三维地物模型；平台 (platform) 与扫描器 (scanner) 目录文件定义飞行平台与扫描仪的物理

参数；扫描任务（survey）文件将场景、平台、扫描器与航迹（trajectory）关联起来，并设定脉冲频率、扫描频率与扫描角等核心参数。

2.2. Web 三维可视化与 Three.js

Three.js 是一个基于 WebGL^[1] 的开源 JavaScript 三维图形库，封装了场景（Scene）、相机（Camera）、渲染器（Renderer）与网格（Mesh）等核心对象，支持 OBJ、GLTF、STL 等多种三维模型格式的加载，并提供了轨道控制（OrbitControls）、拖拽控制（DragControls）与射线拾取（Raycaster）等交互组件。借助 Three.js，可在浏览器中构建轻量级的三维可视化场景，实现模型展示、点云渲染与鼠标交互拾取，为航路规划提供直观的操作界面。

2.3. Vue 3 前端框架

Vue 3 是一个渐进式 JavaScript 框架，采用 Composition API 提供更好的代码组织和类型支持。其核心特性包括响应式系统、组件化开发和虚拟 DOM。本系统前端基于 Vue 3 构建，使用 Pinia 进行状态管理，通过组件化方式组织用户界面，实现了高效的 3D 交互和参数配置功能。

2.4. FastAPI 与后端异步任务

FastAPI 是一个基于 Python 的高性能 Web 框架，基于类型注解与 Pydantic^[2] 实现自动的数据校验与接口文档生成，并原生支持异步编程与 WebSocket。本系统后端基于 FastAPI 构建 REST API^[3] 与 WebSocket 日志通道，通过 asyncio 子进程异步调用 HELIOS++ 可执行文件，实时捕获其标准输出并以 WebSocket 推送到前端，实现仿真过程的实时监控。

3. 系统需求分析与设计

3.1. 用户需求分析

本系统的目标用户为航测工程师、遥感研究人员以及相关专业师生。其核心需求可归纳为三个方面：一是直观的航路规划，用户希望在不编写配置文件的情况下，通过三维场景中的鼠标交互即可完成航点设定与航线编辑；二是快速的仿真执行，用户能够通过图形化界面配置平台与扫描仪参数，一键调用 HELIOS++ 引擎并实时查看运行状态；三是便捷的结果可视化，仿真生成的点云能够自动加载到三维场景中，并支持按高度或强度着色以及尺寸、透明度的调节。

3.2. 系统总体架构

系统采用浏览器/服务器（B/S）架构，分为前端、后端与仿真引擎三层，其总体架构如图1所示。前端基于 Vue 3 构建，负责三维场景渲染、航点交互编辑、参数配置与结果展示，通过 HTTP^[4-6] 与 WebSocket 与后端通信；后端基于 FastAPI，负责接收前端请求，生成 HELIOS++ 所需的航迹文件与 XML 配置文件，调用 HELIOS++ 可执行文件执行仿真，并解析与返回点云结果；仿真引擎层为 HELIOS++，独立于前后端，由后端以子进程方式调用。各模块间通过明确的 REST API 与 WebSocket 消息格式进行数据交换，形成完整的工作流。

3.3. 功能优先级划分

结合开发环境与工程约束，系统功能划分为三个优先级：核心功能（MVP）包括三维场景渲染、模型加载、交互式航点编辑、航迹导出、仿真参数配置、仿真执行、日志控制台与点云渲染；重要功能包

括场景要素属性编辑、自动航线生成、航高自动计算、仿真预设方案、点云特征统计与点云下载；拓展功能包括仿真进度实时反馈、点云覆盖度分析、多任务队列管理与仿真过程动画。

4. 系统设计与实现

4.1. 系统模块划分

系统实现按前后端分层组织，前端基于 Vue 3 与 Node.js^[7] 生态构建，目录下包含组件（components）、状态管理（stores）与组合式函数（composables）三部分：组件层包含三维场景 Scene3D.vue、参数配置面板 ControlPanel.vue、航点列表 WaypointList.vue 与日志控制台 LogConsole.vue；stores 层基于 Pinia 维护场景（scene）、航点（waypoints）与仿真（simulation）三份响应式状态；composables 层封装 Three.js 场景管理（useThreeScene）、航点交互（useWaypoints）与后端接口调用（useHeliosAPI）。

后端基于 FastAPI 构建，api 目录下包含 REST 路由 routes.py 与 WebSocket 端点 websocket.py，models 目录定义 Pydantic 数据模型 schemas.py，services 目录封装航迹生成 trajectory_generator.py、配置生成 config_generator.py、HELIOS++ 调用 helios_service.py 与点云解析 pointcloud_parser.py。

4.2. 三维场景渲染与航点交互

三维场景基于 Three.js 构建，Scene3D.vue 负责初始化场景、相机、渲染器与灯光，并添加地面网格与拾取平面。场景使用 Z-up 坐标系（与 HELIOS++ 一致，z 为高程轴），通过 OrbitControls 实现相机的旋转、平移与缩放；模型加载采用 OBJLoader、GLTFLoader 与 STLLoader，加载完成后根据模型包围盒自动调整相机视角，并基于模型顶点包围盒推断其 up 轴（Y-up 模型自动旋转至 Z-up）。航点交互通过自绘拖拽实现：鼠标左键点击经 Raycaster 与地面平面求交，在交点处添加球体表示的航点（恒为 z=0 贴地），相邻航点之间以白色折线连接；拖拽航点时锁定相机（‘controls.enabled=false’），确保不干扰视角操作；通过 Delete 键或右键菜单删除航点。

4.3. 前端组件与状态管理

侧边栏采用多 Tab 布局，包含仿真参数、点云、模型列表、航迹与设置五个 Tab。ControlPanel.vue 提供仿真参数表单，分为「载体参数」与「传感器参数」两个模块，各参数旁显示有效范围，只读参数以灰色禁用显示；参数数据来源于从 HELIOS++ 扫描器与平台目录文件中提取的规格库，切换平台类型时自动重置参数至默认值。WaypointList.vue 以列表形式展示航点序号与坐标，支持单个删除与整体清空。LogConsole.vue 展示分级日志（INFO/WARNING/ERROR），支持自动滚动与清空。

三个 Pinia store 分别维护场景模型与点云渲染参数（scene）、航点坐标数组（waypoints）以及仿真任务状态、进度、日志与结果（simulation）。前端通过 useHeliosAPI.js 中的 Axios 实例调用后端 REST 接口，并通过 connectLogWS 建立 WebSocket 连接以接收仿真日志。

4.4. 后端 API 接口设计

后端基于 FastAPI 实现，提供完整的 RESTful API 接口，主要分为以下六类：

1. **模型管理**：上传/列表/删除三维模型，支持 OBJ、GLTF、GLB、STL 格式
2. **航迹管理**：从航点生成 HELIOS++ 原生.trj 航迹文件，支持弓字形自动生成
3. **配置管理**：生成 HELIOS++ survey XML 与 scene XML 配置文件
4. **仿真执行**：异步调用 HELIOS++ 子进程，实时推送日志，支持状态查询与取消

- 5. **结果管理**：返回点云统计（点数/边界/高程/强度/直方图）、渲染抽样与文件下载
- 6. **环境诊断**：检测 HELIOS++ 可执行文件、资源目录、静态工作目录的就绪状态

表 1: FeHALS REST API 接口列表

接口组	HTTP 方法	功能说明
模型管理	GET /api/models	获取所有上传的模型列表
	POST /api/models/upload	上传 3D 模型文件
	DELETE /api/models/{id}	删除指定模型
航迹管理	POST /api/trajectory/generate	从航点生成航迹文件
	GET /api/trajectories	获取所有航迹列表
	GET /api/trajectories/download/{id}	下载航迹文件
配置管理	POST /api/config/generate	生成仿真配置文件
仿真执行	POST /api/simulation/run	执行 HELIOS++ 仿真
	GET /api/simulation/status/{task_id}	获取仿真状态
	GET /api/simulation/logs/{task_id}	获取仿真日志
	POST /api/simulation/cancel/{task_id}	取消仿真任务
结果管理	GET /api/results/{task_id}	获取点云数据与统计
	GET /api/results/{task_id}/download	下载原始结果
环境诊断	GET /api/env/diagnose	检查系统环境
缓存管理	GET /api/cache	查看各缓存目录大小
	DELETE /api/cache/{type}	分类清理缓存

4.5. 航迹与配置文件生成

trajectory_generator.py 将航点数组转换为 HELIOS++ 原生航迹文件 (.trj, CSV 格式, 列顺序为时间 t、roll、pitch、yaw、x、y、z)，所有航点统一为恒定飞行高度（仅用 x、y 定义水平路径）。在航迹生成过程中，系统按以下步骤计算航迹点：

- 偏航角计算**：从当前航点至下一航点的方向向量经 $yaw = \arctan(\Delta x / \Delta y)$ 转换为 ARINC 705 规范的偏航角， $yaw=0^\circ$ 表示正北方向 (+Y)， $yaw=90^\circ$ 表示正东方向 (+X)，确保扫描线始终正交于航线方向。
- 时间步长计算**：根据目标飞行速度（默认 10 m/s）和航段距离计算各航段的时间步长 $dt = \max(1.0, dist/v_{target})$ ，时间沿航线累积，使各航段的速度连续且物理合理。
- 转角瞬时转向**：在航点转角处，系统生成同一时间戳、同一位置、不同偏航角的两行记录，第一行保持上一段的偏航角，第二行切换至下一段的偏航角，实现瞬时转向，避免偏航角在航段间渐变造成扫描线偏移。

系统还支持弓字形自动航迹生成：用户在三维场景中点击两个角点定义矩形区域，系统根据设定航线间距自动生成往返覆盖的全覆盖航线，相邻扫描线的 X 端点次序交替，形成连续往返路径。扫描条带宽度由 $W = 2 \cdot h \cdot \tan(\theta/2)$ 计算，扫描线间距由 $S = v/f_{scan}$ 计算，其中 h 为飞行高度， θ 为总扫描角， v 为飞行速度， f_{scan} 为扫描频率。

config_generator.py 负责生成 HELIOS++ 所需的两个 XML 文件：场景文件通过 objloader 滤镜引用三维模型，并指定模型的 up 轴 (Y-up 自动旋转至 Z-up)；扫描任务 (survey) 文件引用场景、平台与扫描器目录，并将平台类型映射为对应的平台目录项 (UAV 与 Airborne 均使用 ‘copter_linearpath’ 平台、‘riegl_vux-1uav’ 扫描器)，将脉冲频率 (kHz) 与扫描角 ($\pm$ 半角) 换算为 HELIOS++ 所需的脉冲频率 (Hz) 与总扫描角。

4.6. HELIOS++ 调用与点云渲染

helios_service.py 维护仿真任务注册表，通过 asyncio.create_subprocess_exec 以子进程方式调用 helios++ 可执行文件，指定资源目录 (-assets)、输出目录 (-output) 与输出格式参数，实时捕获其标准输出/错误流并解析进度百分比，经 WebSocket 推送至前端，同时支持超时终止与任务取消。仿真启动前，前端对全部参数进行范围校验（数据来源于从 HELIOS++ 扫描器目录文件中提取的参数规格库），超出有效范围时拒绝执行并给出提示。系统还提供 HELIOS++ 运行环境诊断功能，通过 GET /api/env/diagnose 接口检测可执行文件存在性、资源目录完整性和静态工作目录就绪状态，在前端设置面板中集中展示。

仿真结束后，pointcloud_parser.py 利用 laspy^[8] 解析 LAS/LAZ 点云或按文本方式解析 XYZ 点云，提取坐标与强度、计算空间范围。点云统计信息包括总点数、三维边界、高程均值、标准差、最小/最大、中位数、P5/P95 分位数，以及 40 组高程分布直方图。强度字段存在时同样计算其均值、标准差和范围。对于超过百万级的点云，使用覆盖全文件序列的等间隔索引抽样进行渲染，全量统计基于完整点集计算，确保统计准确性。前端将返回的点云坐标构建为 BufferGeometry 与 Points 对象，支持按高度（蓝色至红色渐变）、按强度或固定颜色着色，并可实时调节点尺寸与透明度。

4.7. 项目组织结构

Fig. 2: FeHALS 项目目录结构

```
1 FeHALS/
2 +-- backend/           # 后端代码
3 |   +-- app/
4 | |   +-- main.py      # FastAPI应用入口
5 | |   +-- config.py    # 配置管理
6 | |   +-- api/         # API路由
7 | |   +-- models/      # 数据模型
8 | |   +-- services/    # 业务服务
9 | |   +-- static/      # 静态文件目录
10 | +-- requirements.txt  # Python依赖
11 | +-- run.py           # 后端启动脚本
12 +-- frontend/         # 前端代码
13 |   +-- src/
14 | |   +-- components/  # Vue组件
15 | |   +-- composables/ # 组合式函数
16 | |   +-- stores/      # Pinia状态管理
17 | |   +-- assets/      # 静态资源
18 | +-- package.json
19 | +-- vite.config.js
20 +-- doc/              # 文档目录
21 |   +-- src/
22 | |   +-- DESCRIPTION.tex
23 | |   +-- DESIGNBOOK.tex
24 | |   +-- DOCUMENT.tex
25 | +-- Makefile        # 构建脚本
26 +-- run.sh            # Unix启动脚本
```

5. 部署与运行指南

5.1. 环境要求

表 2: 系统环境要求

组件	要求
Node.js	>= 16.0.0
Python	>= 3.8.0
npm	>= 7.0.0
LaTeX	XeLaTeX (用于文档编译)

5.2. 安装步骤

1. 后端安装

```
1 # 1. 进入后端目录
2 cd backend
3
4 # 2. 创建虚拟环境
5 python -m venv venv
```

```

6
7 # 3. 激活虚拟环境
8 # Linux/Mac
9 source venv/bin/activate
10 # Windows
11 venv\Scripts\activate
12
13 # 4. 安装依赖
14 pip install -r requirements.txt
15
16 # 5. 设置环境变量
17 export HELIOS_PATH=/path/to/helios++
18 export HELIOS_REPO=/path/to/helios++/repo
19 export HELIOS_ASSETS=/path/to/assets

```

2. 前端安装

```

1 # 1. 进入前端目录
2 cd frontend
3
4 # 2. 安装依赖
5 npm install
6
7 # 3. 启动开发服务器
8 npm run dev

```

5.3. 启动脚本

项目提供了启动脚本用于快速启动服务：

```

1 #!/bin/bash
2
3 # 停止所有服务
4 stop_services() {
5     echo "停止所有服务..."
6     pkill -f "uvicorn" || true
7     pkill -f "vite" || true
8 }
9
10 # 启动后端
11 start_backend() {
12     echo "启动后端服务..."
13     cd backend
14     source venv/bin/activate
15     python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000 &
16     cd ..
17 }
18
19 # 启动前端
20 start_frontend() {
21     echo "启动前端服务..."
22     cd frontend
23     npm run dev &
24     cd ..

```

```

25 }
26
27 # 主程序
28 case "$1" in
29     start)
30         start_backend
31         start_frontend
32         echo "服务已启动"
33         echo "前端:␣http://localhost:3000"
34         echo "后端:␣http://localhost:8000"
35         echo "API文档:␣http://localhost:8000/docs"
36         ;;
37     stop)
38         stop_services
39         echo "服务已停止"
40         ;;
41     *)
42         echo "用法:␣$0{start|stop}"
43         exit 1
44         ;;
45 esac

```

Listing 1: run.sh 脚本

5.4. 配置说明

• 后端配置

```

1  # 环境变量示例
2  export FEHALS_HOST=0.0.0.0
3  export FEHALS_PORT=8000
4  export HELIOS_PATH=/usr/local/bin/helios++
5  export HELIOS_REPO=/opt/helios++
6  export HELIOS_ASSETS=/opt/helios++/assets
7  export FEHALS_SIM_TIMEOUT=300
8  export FEHALS_CORS_ORIGINS=http://localhost:3000,http://127.0.0.1:3000

```

• 前端配置

```

1  // vite.config.js
2  export default defineConfig({
3      server: {
4          proxy: {
5              '/api': {
6                  target: 'http://localhost:8000',
7                  changeOrigin: true
8              },
9              '/ws': {
10                 target: 'ws://localhost:8000',
11                 ws: true
12             }
13         }
14     }
15 })

```

6. API 接口详解

6.1. REST API 接口

6.1.1. 模型管理接口

模型管理接口支持三维模型的增删查操作。请求与响应格式如下：

表 3: 模型管理 API 接口

接口路径	方法	请求体	响应
/api/models	GET	无	模型列表 (id/name/url/up)
/api/models/upload	POST	multipart: file	{model_id, filename, url, up}
/api/models/{id}	DELETE	路径参数 {id}	{success: true}

上传模型接口示例

```
# 请求
curl -X POST http://localhost:8000/api/models/upload \
-F "file=@building.obj"

# 响应
{"model_id": "mod_1700000000_abc123", "filename": "building.obj",
 "url": "/static/models/mod_1700000000_abc123.obj", "up": "z"}
```

Listing 2: 上传 3D 模型

获取模型列表

```
# 请求
curl http://localhost:8000/api/models

# 响应
{"models": [
  {"id": "mod_...", "name": "building.obj", "url": "/static/models/...", "up": "z"}
]}
```

Listing 3: 获取模型列表

6.1.2. 航迹管理接口

航迹管理接口从航点生成 HELIOS++ 原生.trj 文件，支持下载。

表 4: 航迹管理 API 接口

接口路径	方法	请求体	响应
/api/trajectory/generate	POST	{waypoints: [[x,y,z],...], altitude}	{file_id, download_url, point_count}
/api/trajectories	GET	无	{trajectories: [id, created]}
/api/trajectories/download/{id}	GET	路径参数 {id}	.trj 文件（文本/纯文本）

生成航迹接口示例

```
# 请求：三个航点构成L形航线
curl -X POST http://localhost:8000/api/trajectory/generate \
```

```
3 -H "Content-Type: application/json" \
4 -d '{"waypoints":[[0,0,0],[100,0,0],[100,100,0]],"altitude":100}'
5
6 # 响应
7 {"file_id":"traj_...", "download_url":"/api/trajectories/download/traj_...",
8  "point_count":3}
9
10 # 生成的.trj文件内容 (CSV)
11 # 0.00,0,0,0.00,0.000000,0.000000,100.000000
12 # 10.00,0,0,0.00,100.000000,0.000000,100.000000
13 # 10.00,0,0,90.00,100.000000,0.000000,100.000000
14 # 20.00,0,0,90.00,100.000000,100.000000,100.000000
```

Listing 4: 生成航迹

6.1.3. 仿真执行接口

仿真执行接口异步调用 HELIOS++ 引擎，任务状态通过轮询或 WebSocket 获取。

表 5: 仿真执行 API 接口

接口路径	方法	请求体	响应
/api/simulation/run	POST	{trajectory_id, config_id, scene_model_id}	{task_id}
/api/simulation/status/{task_id}	GET	路径参数 {task_id}	{status, progress, message, result_file}
/api/simulation/logs/{task_id}	GET	路径参数 {task_id}	{logs: [string]}
/api/simulation/cancel/{task_id}	POST	路径参数 {task_id}	{success: bool}

启动仿真接口示例

```
1 # 1. 先上传模型、生成航迹、生成配置，获得各ID
2 # 2. 提交仿真任务
3 curl -X POST http://localhost:8000/api/simulation/run \
4 -H "Content-Type: application/json" \
5 -d '{"trajectory_id":"traj_...", "config_id":"cfg_...",
6     "scene_model_id":"mod_..."}'
7
8 # 响应
9 {"task_id":"sim_..."}
10
11 # 查询状态
12 curl http://localhost:8000/api/simulation/status/sim_...
13 # {"task_id":"sim_...", "status":"running", "progress":45, "message":"","result_file":null}
```

Listing 5: 启动仿真

6.1.4. 结果管理接口

仿真完成后，结果接口返回点云统计与渲染抽样数据。

获取结果接口示例

```
1 curl http://localhost:8000/api/results/sim_...
2
3 # 响应
```

表 6: 结果管理 API 接口

接口路径	方法	参数	响应
/api/results/{task_id}	GET	路径参数 {task_id}	{file_path, point_count, bounds, stats, point
/api/results/{task_id}/download	GET	路径参数 {task_id}, ?format=las	原始结果文件

```
4 {"file_path":"/static/results/sim_.../output.xyz",
5  "point_count":125000,
6  "bounds":{"minx":-10.5,"miny":-15.2,"minz":85.3,
7           "maxx":110.8,"maxy":115.6,"maxz":102.1},
8  "stats":{"mean_z":93.7,"std_z":4.2,"min_z":85.3,"max_z":102.1,
9           "median_z":94.1,"p5_z":88.2,"p95_z":100.5,
10          "histogram":[[85,100],[86,250],...]},
11  "points":{"x":[...],"y":[...],"z":[...],"intensity":[...]}}
```

Listing 6: 获取结果

6.1.5. 环境诊断与缓存管理

表 7: 环境诊断与缓存管理 API

接口路径	方法	参数	响应
/api/env/diagnose	GET	无	{helios_status, assets_status, dirs_status}
/api/cache	GET	无	{cache: {models: {count, size}, ...}}
/api/cache/{type}	DELETE	路径参数 {type}	{success, removed}

6.2. WebSocket 接口

6.2.1. 日志推送

仿真任务运行期间，后端通过 WebSocket 向 /ws/logs/{task_id} 推送实时日志。

WebSocket 连接示例

```
1 // 前端 useHeliosAPI.js 中的 WebSocket 连接逻辑
2 const ws = new WebSocket(`ws://${host}/ws/logs/${taskId}`);
3 ws.onmessage = (evt) => {
4   const msg = JSON.parse(evt.data);
5   switch (msg.type) {
6     case 'log': console.log(`[${msg.level}] ${msg.message}`); break;
7     case 'status': updateStatus(msg.status, msg.progress); break;
8     case 'result': loadResult(msg.task_id); break;
9   }
10 };
```

Listing 7: WebSocket 客户端连接

6.3. 数据模型定义

后端的请求与响应数据结构由 Pydantic 模型定义,源文件位于 backend/app/models/schemas.py:

表 8: WebSocket 日志消息格式

消息类型	数据字段	说明
log	timestamp	日志时间戳
	level	日志级别 (INFO/WARNING/ERROR)
	message	日志消息内容
	progress	进度百分比 (0-100)
status	task_id	任务 ID
	status	任务状态 (running/completed/failed/cancelled)
	progress	当前进度
	error	错误信息 (如果有)
result	task_id	任务 ID
	result_path	结果文件路径
	point_count	点云总数
	file_size	结果文件大小

```
1  """Pydantic 数据模型 (请求/响应 schema)。"""
2  from typing import List, Optional
3
4  from pydantic import BaseModel
5
6
7  class TrajectoryRequest(BaseModel):
8      """航迹生成请求: 航点列表+飞行高度。"""
9
10     waypoints: List[List[float]] # [[x, y, z], ...]
11     altitude: float = 100.0
12
13
14     class CoverageRequest(BaseModel):
15         """点云覆盖度分析请求: 点云坐标+网格分辨率。"""
16
17         points: List[List[float]] # [[x, y, z], ...]
18         grid_size: int = 50
19
20
21     class ConfigRequest(BaseModel):
22         """仿真参数配置请求。"""
23
24         platform_id: str = "copter_linearpath"
25         scanner_id: str = "riegl_vux-luav"
26         speed: float = 5.0 # 飞行速度 (m/s)
27         altitude: float = 100.0 # 飞行高度 (m)
28         scan_freq: float = 10.0 # 扫描频率 (Hz)
29         scan_angle: float = 30.0 # 扫描角度范围 (±deg, 半角)
30         pulse_freq: float = 50.0 # 脉冲频率 (kHz)
31         # 默认 XYZ: 本机 helios++ 构建的 LAS 输出不可用 (LASlib 打开失败), 见 README。
32         # LAS/LAZ 可选, 需 HELIOS++ 正确链接 LASlib 后方可使用。
```

```

33     output_format: str = "XYZ" # LAS | LAZ | XYZ
34
35
36 class SimulationRunRequest(BaseModel):
37     """仿真执行请求。"""
38
39     trajectory_id: str
40     config_id: str
41     scene_model_ids: Optional[List[str]] = None
42
43
44 class SimulationStatus(BaseModel):
45     """仿真状态。"""
46
47     task_id: str
48     status: str # pending | running | completed | failed
49     progress: int = 0
50     message: str = ""
51     result_file: Optional[str] = None

```

Listing 8: backend/app/models/schemas.py

7. 前端实现详解

7.1. Vue 3 应用架构

7.1.1. 组件结构

前端采用 Vue 3 Composition API 构建，主要组件包括：

```

1 frontend/src/
2 +-- components/          # Vue 组件
3 |   +-- Scene3D.vue      # 3D场景渲染组件
4 |   +-- ControlPanel.vue # 参数配置面板
5 |   +-- WaypointList.vue # 航点列表组件
6 |   +-- LogConsole.vue   # 日志控制台
7 |   +-- ModelList.vue    # 模型管理组件
8 |   +-- PointCloudPanel.vue # 点云渲染面板
9 |   +-- SettingsPanel.vue # 设置面板
10 +-- stores/              # Pinia状态管理
11 |   +-- scene.js         # 场景状态
12 |   +-- waypoints.js     # 航点状态
13 |   +-- simulation.js    # 仿真状态
14 +-- composables/         # 组合式函数
15     +-- useThreeScene.js # Three.js场景管理
16     +-- useWaypoints.js  # 航点交互逻辑
17     +-- useHeliosAPI.js  # API调用封装
18     +-- useBowtie.js     # 弓字形航线生成

```

Listing 9: 前端组件结构

7.1.2. 3D 场景组件实现

以下为 Scene3D.vue 的核心实现，包含场景初始化、模型加载与航点渲染的调度逻辑：

```

1 <script setup>
2 import { onMounted, onBeforeUnmount, ref, watch } from 'vue'
3 import { useThreeScene } from '../composables/useThreeScene'
4 import { useWaypoints } from '../composables/useWaypoints'
5 import { useSimAnimation } from '../composables/useSimAnimation'
6 import { useSceneStore } from '../stores/scene'
7 import { useWaypointStore } from '../stores/waypoints'
8 import { useSimulationStore } from '../stores/simulation'
9 import { useAnimationStore } from '../stores/animation'
10 import PlaybackBar from './PlaybackBar.vue'
11
12 const three = useThreeScene()
13 const waypoints = useWaypoints()
14 const anim = useSimAnimation()
15 const sceneStore = useSceneStore()
16 const waypointStore = useWaypointStore()
17 const simStore = useSimulationStore()
18 const animStore = useAnimationStore()
19
20 const container = ref(null)
21 const loadedIds = new Set()
22
23 // ----- 仿真动画接线 -----
24
25 // 播放状态快照：动画驱动器每帧读取，天然与 store 保持同步（无需逐项 watch）
26 function playbackSnapshot() {
27   return {
28     playing: animStore.playing,
29     progress: animStore.progress,
30     speed: animStore.speed,
31     loop: animStore.loop,
32     duration: animStore.stats.duration,
33     revealMode: animStore.revealMode,
34     showPlatform: animStore.showPlatform,
35     showTrail: animStore.showTrail,
36     followCamera: animStore.followCamera,
37     platformScale: animStore.platformScale,
38     platformType: simStore.params.platform_id,
39     altitude: simStore.params.altitude,
40     pointCount: three.getPointCloudCount(),
41   }
42 }
43
44 // 航点 + 航高 → 飞行航迹（与后端 .trj 生成规则一致：恒定航高，仅取航点 x、y）
45 function rebuildCourse() {
46   const pts = waypointStore.points
47   const altitude = Number(simStore.params.altitude) || 0
48   const speed = Number(simStore.params.speed) > 0 ? Number(simStore.params.speed) : 1
49   const { length, segments } = anim.setCourse(pts, altitude)
50   animStore.ready = pts.length > 0
51   animStore.setStats({
52     pathLength: length,
53     segments,

```

```

54     duration: length / speed, // 单倍速播放时长 = 航迹长度 / 飞行速度
55     altitude,
56     total: three.getPointCloudCount(),
57   })
58   if (!animStore.ready) animStore.reset()
59 }
60
61 onMounted(() => {
62   three.init(container.value)
63
64   waypoints.setCallbacks({
65     onAdd: (p) => waypointStore.add(p),
66     onMove: (index, p) => waypointStore.update(index, p),
67     onRemove: (index) => waypointStore.remove(index),
68   })
69   waypoints.renderWaypoints(waypointStore.points)
70
71   anim.mount({
72     getPlayback: playbackSnapshot,
73     onProgress: (progress, info) =>
74       animStore.updateProgress(progress, info.revealed, info.total, info.finished),
75   })
76   rebuildCourse()
77 })
78
79 // 动画总开关（设置 Tab）：关闭时卸载驱动器并隐藏播放条，开启时重新挂载并重建航迹
80 watch(
81   () => animStore.enabled,
82   (on) => {
83     if (!on) {
84       anim.unmount()
85       animStore.reset()
86       return
87     }
88     anim.mount({
89       getPlayback: playbackSnapshot,
90       onProgress: (progress, info) =>
91         animStore.updateProgress(progress, info.revealed, info.total, info.finished),
92     })
93     rebuildCourse()
94   }
95 )
96
97 onBeforeUnmount(() => {
98   anim.unmount()
99   three.dispose()
100 })
101
102 // store 航点变化 → 重建三维表示
103 watch(
104   () => waypointStore.points,
105   (points) => waypoints.renderWaypoints(points),
106   { deep: true }
107 )

```

```

108
109 // 航点/航高/速度变化 → 重建飞行航迹与动画统计
110 watch(
111   () => [
112     waypointStore.points,
113     simStore.params.altitude,
114     simStore.params.speed,
115   ],
116   () => rebuildCourse(),
117   { deep: true }
118 )
119
120 // 模型变化 → 加载到场景, 或从场景移除
121 watch(
122   () => sceneStore.models.map((m) => m.id),
123   (ids) => {
124     // 移除已加载但 store 中不再存在的模型
125     for (const id of loadedIds) {
126       if (!ids.includes(id)) {
127         three.removeModel(id)
128         loadedIds.delete(id)
129       }
130     }
131     // 加载新模型
132     sceneStore.models.forEach((m) => {
133       if (loadedIds.has(m.id)) return
134       loadedIds.add(m.id)
135       sceneStore.setLoading(true)
136       three
137         .loadModel(m.url, m.name, m.up, m.id)
138         .then(({ size }) => {
139           sceneStore.setLoading(false)
140           sceneStore.setModelBbox(m.id, { size })
141         })
142         .catch((err) => {
143           sceneStore.setLoading(false)
144           simStore.addLog('ERROR', `模型加载失败 (${m.name}) : ${err.message || err}`)
145         })
146     })
147   },
148   { deep: true }
149 )
150
151 // 模型可见性
152 watch(
153   () => sceneStore.models.map((m) => ({ id: m.id, visible: m.visible })),
154   (vals) => vals.forEach((v) => three.setModelVisible(v.id, v.visible)),
155   { deep: true }
156 )
157
158 // 模型 bbox
159 watch(
160   () => sceneStore.models.map((m) => ({ id: m.id, showBbox: m.showBbox })),
161   (vals) => vals.forEach((v) => three.setModelBbox(v.id, v.showBbox)),

```

```

162   { deep: true }
163 )
164
165 // 仿真结果 → 点云渲染 + 动画回放
166 watch(
167   () => simStore.result,
168   (result) => {
169     if (!result) return
170     three.setPointCloud(result.points, result.intensity, sceneStore.pointOptions)
171     animStore.setStats({ total: three.getPointCloudCount() })
172     // 仿真回放关闭时：仅加载点云并完整呈现，不与原有结果加载流程耦合
173     if (!animStore.enabled) {
174       three.revealPointCloud(Infinity)
175       return
176     }
177     // 自动播放开启时从头演示点云生成过程；否则直接呈现完整点云（进度置 100%）
178     if (animStore.ready && animStore.autoPlay) {
179       animStore.restart()
180     } else {
181       animStore.seek(1)
182       three.revealPointCloud(Infinity)
183     }
184   }
185 )
186
187 // 点云渲染参数 → 实时更新（drawRange 不受影响，动画进度保持不变）
188 watch(
189   () => sceneStore.pointOptions,
190   (opts) => three.updatePointCloud(opts),
191   { deep: true }
192 )
193
194 // 渲染选项变化 → 应用到 Three.js 场景
195 watch(
196   () => sceneStore.renderOptions,
197   (opts) => {
198     three.setAxesVisible(opts.showAxes)
199     three.setWaypointColor(opts.waypointColor)
200     three.setTrajectoryColor(opts.trajectoryColor)
201     three.setArrowColor(opts.arrowColor)
202     three.setArrowsVisible(opts.showArrows)
203   },
204   { deep: true, immediate: true }
205 )
206 </script>
207
208 <template>
209   <div class="scene3d">
210     <div ref="container" class="scene-container"></div>
211
212     <PlaybackBar />
213
214     <div v-if="sceneStore.loading" class="loading-overlay">
215       <div class="spinner"></div>

```

```

216     <span>模型加载中...</span>
217   </div>
218 </div>
219 </template>

```

Listing 10: frontend/src/components/Scene3D.vue

7.2. Three.js 集成与 3D 交互

7.2.1. 场景管理组合式函数

以下为 useThreeScene.js 的核心实现，封装了 Three.js 场景、相机、渲染器、模型加载与航点交互逻辑：

```

1 import * as THREE from 'three'
2 import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'
3 import { GLTFLoader } from 'three/examples/jsm/loaders/GLTFLoader.js'
4 import { OBJLoader } from 'three/examples/jsm/loaders/OBJLoader.js'
5 import { STLLoader } from 'three/examples/jsm/loaders/STLLoader.js'
6
7 // 高度着色渐变（蓝→青→绿→黄→红）
8 // 3D 点云着色（本文件）与统计直方图（PointCloudPanel）共用
9 export function heightColor(t) {
10   const stops = [[0, 0, 1], [0, 1, 1], [0, 1, 0], [1, 1, 0], [1, 0, 0]]
11   const x = Math.max(0, Math.min(1, t)) * (stops.length - 1)
12   const i = Math.min(Math.floor(x), stops.length - 2)
13   const f = x - i
14   const a = stops[i], b = stops[i + 1]
15   return [a[0] + (b[0] - a[0]) * f, a[1] + (b[1] - a[1]) * f, a[2] + (b[2] - a[2]) * f]
16 }
17
18 export function heightCssColor(t) {
19   const [r, g, b] = heightColor(t)
20   return `rgb(${Math.round(r * 255)}, ${Math.round(g * 255)}, ${Math.round(b * 255)})`
21 }
22
23 // 单例场景管理器：负责 Three.js 场景、模型加载、点云渲染与航点交互。
24 let singleton = null
25
26 export function useThreeScene() {
27   if (!singleton) singleton = createThreeScene()
28   return singleton
29 }
30
31 function createThreeScene() {
32   const state = {
33     container: null,
34     renderer: null,
35     scene: null,
36     camera: null,
37     controls: null,
38     raycaster: null,
39     pointer: new THREE.Vector2(),
40     grid: null,
41     groundPlane: null,

```

```

42     modelsGroup: null,
43     waypointGroup: null,
44     waypointLine: null,
45     modelRoots: {}, // id -> THREE.Object3D
46     pendingRemoval: new Set(), // 加载期间被删除的模型 id (丢弃竞态的幽灵对象)
47     bboxHelpers: {}, // id -> THREE.Box3Helper
48     modelMeshes: [], // 参与拾取的模型 Mesh (通过 refreshModelMeshes 重建)
49     waypointSpheres: [], // [{mesh, index}]
50     pointCloud: null,
51     pcData: null, // {points, intensity}
52     selectedIndex: -1,
53     animationId: null,
54     // 仿真动画支撑
55     animGroup: null, // 动画对象根节点 (useSimAnimation 挂载平台/扫描锥/轨迹)
56     clock: null, // 帧间隔计时
57     frameCallbacks: new Set(), // 每帧回调 (外部动画驱动器注册, 共用渲染循环)
58     // 航点交互回调 (由组件注入)
59     wpCallbacks: { onAdd: null, onMove: null, onRemove: null },
60     // 自绘拖拽
61     dragTarget: null, // {mesh, index}
62     dragActive: false,
63     // 矩形选点模式
64     pickMode: 'waypoint',
65     rectPointCallback: null,
66     _downX: 0,
67     _downY: 0,
68     // 自适应网格
69     _lastGridDist: -1,
70     // 可配置渲染属性
71     waypointColor: '#00ffff',
72     waypointSelectColor: '#ffff00',
73     trajectoryColor: '#ff00ff',
74     arrowColor: '#ffff00',
75     waypointArrows: [],
76     axesGroup: null,
77     _lastScaleKey: -1,
78     _baseWpRadius: 0.3,
79     _resizeObserver: null,
80 }
81
82 const CLICK_MOVE_SQ = 25 // 点击 vs 拖动的位移阈值 (像素²)
83
84 // ----- 加粗坐标轴箭头 (Mesh 避免 WebGL linewidth 限制) -----
85
86 function makeAxisArrow(dir, color) {
87     const AXIS_LEN = 12
88     const SHAFT_RAD = 0.10
89     const HEAD_LEN = 1.2
90     const HEAD_RAD = 0.30
91
92     const group = new THREE.Group()
93     const dirVec = new THREE.Vector3(dir[0], dir[1], dir[2])
94     const shaftLen = AXIS_LEN - HEAD_LEN
95

```

```

96     const shaftGeo = new THREE.CylinderGeometry(SHAFT_RAD, SHAFT_RAD, shaftLen, 8)
97     const mat = new THREE.MeshBasicMaterial({ color })
98     const shaft = new THREE.Mesh(shaftGeo, mat)
99     shaft.position.set(0, shaftLen / 2, 0)
100    group.add(shaft)
101
102    const headGeo = new THREE.ConeGeometry(HEAD_RAD, HEAD_LEN, 12)
103    const head = new THREE.Mesh(headGeo, mat)
104    head.position.set(0, shaftLen + HEAD_LEN / 2, 0)
105    group.add(head)
106
107    // 从 +Y 朝向目标方向
108    const up = new THREE.Vector3(0, 1, 0)
109    group.quaternion.setFromUnitVectors(up, dirVec)
110    return group
111  }
112
113  // ----- 初始化 -----
114
115  function init(container) {
116    state.container = container
117    const w = container.clientWidth
118    const h = container.clientHeight
119
120    state.renderer = new THREE.WebGLRenderer({ antialias: true })
121    state.renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2))
122    state.renderer.setSize(w, h)
123    container.appendChild(state.renderer.domElement)
124
125    state.scene = new THREE.Scene()
126    state.scene.background = new THREE.Color(0xe9ecef)
127
128    state.camera = new THREE.PerspectiveCamera(60, w / h, 0.1, 100000)
129    // Z-up: 与 HELIOS++ (及地理空间) 坐标一致, z 为高程轴
130    state.camera.up.set(0, 0, 1)
131    state.camera.position.set(10, 10, 10)
132    state.camera.lookAt(0, 0, 0)
133
134    // 灯光
135    state.scene.add(new THREE.AmbientLight(0xffffff, 0.65))
136    const dir = new THREE.DirectionalLight(0xffffff, 0.85)
137    dir.position.set(20, 40, 20)
138    state.scene.add(dir)
139
140    // 网格与 (不可见) 地面拾取平面 (Z-up: 地面为 z=0 的 XY 平面)
141    state.grid = new THREE.GridHelper(500, 100, 0x999999, 0xd0d0d0)
142    state.grid.rotation.x = Math.PI / 2 // GridHelper 默认在 XZ 平面, 转到 XY 平面
143    state.scene.add(state.grid)
144    // 加粗 XYZ 坐标轴 (Mesh 而非 AxesHelper, 避免过细被 grid 中心线覆盖)
145    state.axesGroup = new THREE.Group()
146    ;[
147      { dir: [1, 0, 0], color: 0xff0000, label: 'X' },
148      { dir: [0, 1, 0], color: 0x00ff00, label: 'Y' },
149      { dir: [0, 0, 1], color: 0x0000ff, label: 'Z' },

```

```

150     ].forEach(({ dir, color }) => {
151         const g = makeAxisArrow(dir, color)
152         state.axesGroup.add(g)
153     })
154     state.axesGroup.visible = false
155     state.scene.add(state.axesGroup)
156     state.groundPlane = new THREE.Mesh(
157         new THREE.PlaneGeometry(100000, 100000),
158         new THREE.MeshBasicMaterial({ visible: false })
159     )
160     // PlaneGeometry 默认即 XY 平面（法向 +Z），无需旋转
161     state.groundPlane.name = 'groundPlane'
162     state.groundPlane.position.set(0, 0, 0)
163     state.scene.add(state.groundPlane)
164
165     state.modelsGroup = new THREE.Group()
166     state.scene.add(state.modelsGroup)
167     state.waypointGroup = new THREE.Group()
168     state.scene.add(state.waypointGroup)
169     // 动画层：仿真动画的所有对象挂在此组下，便于整体显隐与释放
170     state.animGroup = new THREE.Group()
171     state.animGroup.name = 'animGroup'
172     state.scene.add(state.animGroup)
173     state.clock = new THREE.Clock()
174
175     state.waypointLine = new THREE.Line(
176         new THREE.BufferGeometry(),
177         new THREE.LineBasicMaterial({ color: state.trajectoryColor })
178     )
179     state.waypointGroup.add(state.waypointLine)
180
181     state.controls = new OrbitControls(state.camera, state.renderer.domElement)
182     state.controls.enableDamping = true
183     state.controls.dampingFactor = 0.08
184
185     state.raycaster = new THREE.Raycaster()
186
187     // 事件监听
188     const dom = state.renderer.domElement
189     dom.addEventListener('pointerdown', onPointerDown)
190     dom.addEventListener('pointermove', onPointerMove)
191     dom.addEventListener('pointerup', onPointerUp)
192     dom.addEventListener('contextmenu', onContextMenu)
193     window.addEventListener('keydown', onKeyDown)
194     state._resizeObserver = new ResizeObserver(() => onResize())
195     state._resizeObserver.observe(state.container)
196
197     animate()
198 }
199
200 function dispose() {
201     if (state.animationId) cancelAnimationFrame(state.animationId)
202     state.frameCallbacks.clear()
203     window.removeEventListener('keydown', onKeyDown)

```

```

204     if (state._resizeObserver) {
205         state._resizeObserver.disconnect()
206         state._resizeObserver = null
207     }
208     if (state.renderer) {
209         const dom = state.renderer.domElement
210         dom.removeEventListener('pointerdown', onPointerDown)
211         dom.removeEventListener('pointermove', onPointerMove)
212         dom.removeEventListener('pointerup', onPointerUp)
213         dom.removeEventListener('contextmenu', onContextMenu)
214         state.controls.dispose()
215         state.renderer.dispose()
216         dom.remove()
217     }
218 }
219
220 function animate() {
221     state.animationId = requestAnimationFrame(animate)
222     // 帧回调先于渲染执行; RAF 已在上方登记, 回调抛错不会中断渲染循环
223     if (state.frameCallbacks.size) {
224         const dt = state.clock ? state.clock.getDelta() : 0
225         state.frameCallbacks.forEach((cb) => cb(dt))
226     }
227     state.controls.update()
228     updateGrid()
229     state.renderer.render(state.scene, state.camera)
230 }
231
232 // 注册/注销每帧回调: cb(dt), dt 为上一帧到本帧的秒数
233 function onFrame(cb) {
234     if (typeof cb === 'function') state.frameCallbacks.add(cb)
235 }
236
237 function offFrame(cb) {
238     state.frameCallbacks.delete(cb)
239 }
240
241 function onResize() {
242     if (!state.renderer || !state.container) return
243     const w = state.container.clientWidth
244     const h = state.container.clientHeight
245     state.camera.aspect = w / h
246     state.camera.updateProjectionMatrix()
247     state.renderer.setSize(w, h)
248     if (state.waypointLine && state.waypointLine.material.resolution) {
249         state.waypointLine.material.resolution.set(w, h)
250     }
251 }
252
253 // ----- 自适应网格 -----
254
255 function updateGrid() {
256     if (!state.controls || !state.grid) return
257     const target = state.controls.target

```

```

258     const dist = state.camera.position.distanceTo(target)
259     // 量化距离，避免每帧重建几何体
260     const gridDist = Math.max(1, Math.round(dist / 10) * 10)
261     if (gridDist !== state._lastGridDist) {
262         state._lastGridDist = gridDist
263         const size = Math.max(30, dist * 2.5)
264         const divisions = Math.max(10, Math.min(150, Math.round(size / 2)))
265         const evenDiv = Math.round(divisions / 2) * 2 // 保证偶数，使中心加粗轴线正确绘制
266         state.scene.remove(state.grid)
267         state.grid = new THREE.GridHelper(size, evenDiv, 0x999999, 0xd0d0d0)
268         state.grid.rotation.x = Math.PI / 2 // Z-up
269         state.scene.add(state.grid)
270     }
271     // 网格跟随视图中心（投影在 z=0 平面）
272     state.grid.position.set(target.x, target.y, 0)
273     // 地面平面也要跟随视图中心，以确保射线拾取与网格显示一致
274     state.groundPlane.position.set(target.x, target.y, 0)
275
276     // 坐标轴跟随网格中心，略抬升避免被 grid 线条遮挡
277     if (state.axesGroup) {
278         state.axesGroup.position.set(target.x, target.y, 0.05)
279     }
280
281     // 缩放航点、箭头和坐标轴以匹配相机距离
282     const scaleKey = Math.round(dist / 3)
283     if (scaleKey !== state._lastScaleKey) {
284         state._lastScaleKey = scaleKey
285         const sf = Math.max(0.3, dist * 0.06)
286         const sf_wps = sf / 2
287         const sf_wpa = sf / 1.5
288         const sf_axes = sf / 5
289         state.waypointSpheres.forEach((s) => {
290             s.mesh.scale.set(sf_wps, sf_wps, sf_wps)
291         })
292         state.waypointArrows.forEach((a) => {
293             a.scale.set(sf_wpa, sf_wpa, sf_wpa)
294         })
295         if (state.axesGroup) {
296             state.axesGroup.scale.set(sf_axes, sf_axes, sf_axes)
297         }
298     }
299 }
300
301 // ----- 拾取 -----
302
303 function setPointer(event) {
304     const rect = state.renderer.domElement.getBoundingClientRect()
305     state.pointer.x = ((event.clientX - rect.left) / rect.width) * 2 - 1
306     state.pointer.y = -((event.clientY - rect.top) / rect.height) * 2 + 1
307     state.raycaster.setFromCamera(state.pointer, state.camera)
308 }
309
310 function raycastGround(event) {
311     setPointer(event)

```

```

312     return state.raycaster.intersectObjects([state.groundPlane], false)
313 }
314
315 function raycastWaypoints(event) {
316     setPointer(event)
317     const targets = state.waypointSpheres.map((s) => s.mesh)
318     return state.raycaster.intersectObjects(targets, false)
319 }
320
321 // ----- 航点交互（自绘拖拽） -----
322
323 function onPointerDown(e) {
324     state._downX = e.clientX
325     state._downY = e.clientY
326     state.dragActive = false
327     // 检查是否命中航点
328     const hits = raycastWaypoints(e)
329     if (hits.length) {
330         const sphere = state.waypointSpheres.find((s) => s.mesh === hits[0].object)
331         if (sphere) {
332             state.dragTarget = { mesh: sphere.mesh, index: sphere.index }
333             state.controls.enabled = false // 锁相机
334         }
335     }
336 }
337
338 function onPointerMove(e) {
339     if (!state.dragTarget) return
340     const dx = e.clientX - state._downX
341     const dy = e.clientY - state._downY
342     if (dx * dx + dy * dy < CLICK_MOVE_SQ) return
343     state.dragActive = true
344     // 射线求交地面平面 z=0, 航点贴地
345     const hits = raycastGround(e)
346     if (hits.length) {
347         const p = hits[0].point
348         state.dragTarget.mesh.position.set(p.x, p.y, 0.05)
349         updateWaypointLine()
350     }
351 }
352
353 function onPointerUp(e) {
354     if (e.button !== 0) return
355     if (state.dragTarget) {
356         if (state.dragActive) {
357             // 拖拽结束 → 推送位置
358             const p = state.dragTarget.mesh.position
359             if (state.wpCallbacks.onMove) {
360                 state.wpCallbacks.onMove(state.dragTarget.index, { x: p.x, y: p.y, z: 0 })
361             }
362         } else {
363             // 点击（无拖拽）→ 选中
364             selectWaypoint(state.dragTarget.index)
365         }
366     }
367 }

```

```

366     state.dragTarget = null
367     state.dragActive = false
368     state.controls.enabled = true
369     return
370 }
371 // 非航点拖拽 → 判断点击 vs 旋转
372 const dx = e.clientX - state._downX
373 const dy = e.clientY - state._downY
374 if (dx * dx + dy * dy > CLICK_MOVE_SQ) return // 旋转/平移, 非点击
375 handleClick(e)
376 }
377
378 function handleClick(e) {
379     if (state.pickMode === 'rect' && state.rectPointCallback) {
380         const hits = raycastGround(e)
381         if (hits.length) {
382             state.rectPointCallback({ x: hits[0].point.x, y: hits[0].point.y, z: 0 })
383         }
384         return
385     }
386     // 先检查是否命中航点 → 选中
387     const wpHits = raycastWaypoints(e)
388     if (wpHits.length) {
389         const sphere = state.waypointSpheres.find((s) => s.mesh === wpHits[0].object)
390         if (sphere) { selectWaypoint(sphere.index); return }
391     }
392     // 命中地面 → 新增航点 (z=0 贴地)
393     const gHits = raycastGround(e)
394     if (gHits.length && state.wpCallbacks.onAdd) {
395         const p = gHits[0].point
396         state.wpCallbacks.onAdd({ x: p.x, y: p.y, z: 0 })
397     }
398 }
399
400 function onContextMenu(e) {
401     e.preventDefault()
402     const hits = raycastWaypoints(e)
403     if (!hits.length) return
404     const sphere = state.waypointSpheres.find((s) => s.mesh === hits[0].object)
405     if (sphere && state.wpCallbacks.onRemove) {
406         state.wpCallbacks.onRemove(sphere.index)
407     }
408 }
409
410 function onKeyDown(e) {
411     if (e.key !== 'Delete' && e.key !== 'Backspace') return
412     const tag = (e.target && e.target.tagName) || ''
413     if (['INPUT', 'TEXTAREA', 'SELECT'].includes(tag)) return
414     if (state.selectedIndex >= 0 && state.wpCallbacks.onRemove) {
415         state.wpCallbacks.onRemove(state.selectedIndex)
416     }
417 }
418
419 function selectWaypoint(index) {

```

```

420     state.selectedIndex = index
421     state.waypointSpheres.forEach((s) => {
422         s.mesh.material.color.set(
423             s.index === index ? state.waypointSelectColor : state.waypointColor
424         )
425     })
426 }
427
428 function setWaypointCallbacks(cb) {
429     state.wpCallbacks = { ...state.wpCallbacks, ...cb }
430 }
431
432 function setPickMode(mode, onRectPoint) {
433     state.pickMode = mode
434     state.rectPointCallback = onRectPoint || null
435 }
436
437 function renderWaypoints(points) {
438     if (state.dragTarget) return // 拖拽中不重建，避免打断交互
439     clearWaypointObjects()
440     const geo = new THREE.SphereGeometry(0.3, 24, 24)
441     points.forEach((p, i) => {
442         const mat = new THREE.MeshStandardMaterial({
443             color: i === state.selectedIndex ? state.waypointSelectColor : state.waypointColor,
444             roughness: 0.4,
445         })
446         const mesh = new THREE.Mesh(geo, mat)
447         mesh.position.set(p.x, p.y, 0.05)
448         mesh.name = 'waypoint'
449         state.waypointGroup.add(mesh)
450         state.waypointSpheres.push({ mesh, index: i })
451     })
452     if (state.selectedIndex >= points.length) state.selectedIndex = -1
453     updateWaypointLine()
454 }
455
456 function clearWaypointObjects() {
457     state.dragTarget = null
458     state.dragActive = false
459     state.waypointSpheres.forEach((s) => {
460         state.waypointGroup.remove(s.mesh)
461         s.mesh.geometry.dispose()
462         s.mesh.material.dispose()
463     })
464     state.waypointSpheres = []
465 }
466
467 function updateWaypointLine() {
468     const positions = []
469     state.waypointSpheres.forEach((s) => {
470         positions.push(s.mesh.position.x, s.mesh.position.y, s.mesh.position.z)
471     })
472     const geo = state.waypointLine.geometry
473     geo.setAttribute('position', new THREE.Float32BufferAttribute(positions, 3))

```

```

474     geo.setDrawRange(0, positions.length / 3)
475     geo.computeBoundingSphere()
476     updateWaypointArrows()
477 }
478
479 function updateWaypointArrows() {
480     // 清除旧箭头
481     state.waypointArrows.forEach((a) => state.waypointGroup.remove(a))
482     state.waypointArrows = []
483     if (!state.waypointSpheres || state.waypointSpheres.length < 2) return
484     for (let i = 0; i < state.waypointSpheres.length - 1; i++) {
485         const p1 = state.waypointSpheres[i].mesh.position
486         const p2 = state.waypointSpheres[i + 1].mesh.position
487         const dir = new THREE.Vector3().subVectors(p2, p1)
488         const len = dir.length()
489         if (len < 0.01) continue
490         dir.normalize()
491         const arrowLen = Math.min(len * 0.3, 2)
492         const arrow = new THREE.ArrowHelper(dir, p1, arrowLen, state.arrowColor, arrowLen * 0.4, arrowLen
            * 0.2)
493         state.waypointGroup.add(arrow)
494         state.waypointArrows.push(arrow)
495     }
496 }
497
498 // ----- 模型 -----
499
500 function loadModel(url, name, up = 'z', id) {
501     return new Promise((resolve, reject) => {
502         if (id) state.pendingRemoval.delete(id) // 新一次加载前清除旧的删除标记
503         const ext = (name.split('.').pop() || '').toLowerCase()
504         let loader
505         if (ext === 'obj') loader = new OBJLoader()
506         else if (ext === 'gltf' || ext === 'glb') loader = new GLTFLoader()
507         else if (ext === 'stl') loader = new STLLoader()
508         else {
509             reject(new Error('不支持的模型格式: ' + ext))
510             return
511         }
512
513         const onLoaded = (obj) => {
514             let root
515             if (ext === 'gltf' || ext === 'glb') root = obj.scene
516             else if (ext === 'stl') {
517                 root = new THREE.Mesh(
518                     obj,
519                     new THREE.MeshStandardMaterial({ color: 0xcccccc, side: THREE.DoubleSide })
520                 )
521             } else root = obj
522
523             if (up === 'y') {
524                 root.rotation.x = Math.PI / 2
525             }
526

```

```

527 // 加载期间模型已被删除：丢弃结果，避免产生幽灵对象
528 if (id && state.pendingRemoval.has(id)) {
529     disposeObject(root)
530     reject(new Error('模型已删除，加载已取消'))
531     return
532 }
533
534 state.modelsGroup.add(root)
535 if (id) state.modelRoots[id] = root
536 refreshModelMeshes()
537 fitViewTo(root)
538 const box = new THREE.Box3().setFromObject(root)
539 const size = box.isEmpty()
540   ? [0, 0, 0]
541   : [Number((box.max.x - box.min.x).toFixed(2)),
542     Number((box.max.y - box.min.y).toFixed(2)),
543     Number((box.max.z - box.min.z).toFixed(2))]
544 resolve({ root, size })
545 }
546 loader.load(url, onLoad, undefined, reject)
547 })
548 }
549
550 function removeModel(id) {
551     state.pendingRemoval.add(id) // 标记：若有进行中的加载，完成后丢弃该模型
552     const root = state.modelRoots[id]
553     if (!root) return
554     state.modelsGroup.remove(root)
555     disposeObject(root)
556     delete state.modelRoots[id]
557     // 清除 bbox + label
558     ;[id, id + '_label'].forEach((k) => {
559         const h = state.bboxHelpers[k]
560         if (h) { state.scene.remove(h); delete state.bboxHelpers[k] }
561     })
562     refreshModelMeshes()
563 }
564
565 function setModelVisible(id, visible) {
566     const root = state.modelRoots[id]
567     if (root) root.visible = visible
568 }
569
570 function setModelBbox(id, show) {
571     const root = state.modelRoots[id]
572     if (!root) return
573     if (show) {
574         if (state.bboxHelpers[id]) return
575         const box = new THREE.Box3().setFromObject(root)
576         const helper = new THREE.Box3Helper(box, 0x00ff00)
577         state.scene.add(helper)
578         state.bboxHelpers[id] = helper
579
580         // 创建尺寸标签 Sprite

```

```

581     const size = box.getSize(new THREE.Vector3())
582     const center = box.getCenter(new THREE.Vector3())
583     const label = makeBboxLabel(size)
584     label.position.set(center.x, center.y, center.z)
585     state.scene.add(label)
586     state.bboxHelpers[id + '_label'] = label
587   } else {
588     const helper = state.bboxHelpers[id]
589     if (helper) { state.scene.remove(helper); delete state.bboxHelpers[id] }
590     const label = state.bboxHelpers[id + '_label']
591     if (label) { state.scene.remove(label); delete state.bboxHelpers[id + '_label'] }
592   }
593 }
594
595 function makeBboxLabel(size) {
596   const text = `${size.x.toFixed(1)} × ${size.y.toFixed(1)} × ${size.z.toFixed(1)}`
597   const canvas = document.createElement('canvas')
598   canvas.width = 256
599   canvas.height = 64
600   const ctx = canvas.getContext('2d')
601   ctx.fillStyle = 'rgba(0,0,0,0.6)'
602   if (ctx.roundRect) ctx.roundRect(0, 0, 256, 64, 8)
603   else ctx.fillRect(0, 0, 256, 64)
604   ctx.fill()
605   ctx.fillStyle = '#00ff00'
606   ctx.font = 'bold 20px Consolas, monospace'
607   ctx.textAlign = 'center'
608   ctx.textBaseline = 'middle'
609   ctx.fillText(text, 128, 32)
610   const tex = new THREE.CanvasTexture(canvas)
611   tex.needsUpdate = true
612   const mat = new THREE.SpriteMaterial({ map: tex, transparent: true, depthTest: false })
613   const sprite = new THREE.Sprite(mat)
614   sprite.scale.set(8, 2, 1)
615   return sprite
616 }
617
618 function clearModels() {
619   Object.keys(state.modelRoots).forEach((id) => removeModel(id))
620   state.modelRoots = {}
621   Object.keys(state.bboxHelpers).forEach((k) => {
622     state.scene.remove(state.bboxHelpers[k])
623   })
624   state.bboxHelpers = {}
625 }
626
627 function refreshModelMeshes() {
628   state.modelMeshes = []
629   Object.values(state.modelRoots).forEach((root) => {
630     root.traverse((o) => { if (o.isMesh) state.modelMeshes.push(o) })
631   })
632 }
633
634 // 场景最高点（所有已加载模型的世界坐标系包围盒并集的最大 Z 值）。

```

```

635 // setFromObject 会应用模型的世界变换 (含 Y-up → Z-up 旋转), 因此结果与 HELIOS++ 高程一致。
636 function getSceneMaxZ() {
637     const box = new THREE.Box3()
638     const tmp = new THREE.Box3()
639     Object.values(state.modelRoots).forEach((root) => {
640         box.union(tmp.setFromObject(root))
641     })
642     return box.isEmpty() ? null : box.max.z
643 }
644
645 function disposeObject(obj) {
646     obj.traverse((o) => {
647         if (o.geometry) o.geometry.dispose()
648         if (o.material) {
649             if (Array.isArray(o.material)) o.material.forEach((m) => m.dispose())
650             else o.material.dispose()
651         }
652     })
653 }
654
655 function fitViewTo(root) {
656     const box = new THREE.Box3().setFromObject(root)
657     if (box.isEmpty()) return
658     const center = box.getCenter(new THREE.Vector3())
659     const size = box.getSize(new THREE.Vector3())
660     const maxDim = Math.max(size.x, size.y, size.z, 0.01)
661     const dist = maxDim * 1.8
662     state.controls.target.copy(center)
663     state.camera.position.set(center.x + dist, center.y + dist, center.z + dist)
664     state.camera.lookAt(center)
665     state.controls.update()
666 }
667
668 // ----- 点云 -----
669
670 function setPointCloud(points, intensity, options) {
671     clearPointCloud()
672     if (!points || !points.length) return
673     state.pcData = { points, intensity }
674
675     const n = points.length
676     const positions = new Float32Array(n * 3)
677     for (let i = 0; i < n; i++) {
678         positions[i * 3] = points[i][0]
679         positions[i * 3 + 1] = points[i][1]
680         positions[i * 3 + 2] = points[i][2]
681     }
682     const geo = new THREE.BufferGeometry()
683     geo.setAttribute('position', new THREE.BufferAttribute(positions, 3))
684
685     const colors = computeColors(points, intensity, options)
686     if (colors) geo.setAttribute('color', new THREE.BufferAttribute(colors, 3))
687
688     const mat = new THREE.PointsMaterial({

```

```

689     size: options.size,
690     vertexColors: !!colors,
691     color: colors ? 0xffffff : new THREE.Color(options.fixedColor || '#ffffff'),
692     transparent: true,
693     opacity: options.opacity,
694     sizeAttenuation: true,
695   })
696
697   state.pointCloud = new THREE.Points(geo, mat)
698   state.scene.add(state.pointCloud)
699   geo.setDrawRange(0, n) // 默认完整显示; 仿真动画用 revealPointCloud 逐步揭示
700 }
701
702 function updatePointCloud(options) {
703   if (!state.pointCloud || !state.pcData) return
704   const mat = state.pointCloud.material
705   mat.size = options.size
706   mat.opacity = options.opacity
707   const colors = computeColors(state.pcData.points, state.pcData.intensity, options)
708   if (colors) {
709     state.pointCloud.geometry.setAttribute('color', new THREE.BufferAttribute(colors, 3))
710     mat.vertexColors = true
711     mat.color.set(0xffffff)
712   } else {
713     state.pointCloud.geometry.deleteAttribute('color')
714     mat.vertexColors = false
715     mat.color.set(options.fixedColor || '#ffffff')
716   }
717   mat.needsUpdate = true
718 }
719
720 function clearPointCloud() {
721   if (state.pointCloud) {
722     state.scene.remove(state.pointCloud)
723     state.pointCloud.geometry.dispose()
724     state.pointCloud.material.dispose()
725     state.pointCloud = null
726     state.pcData = null
727   }
728 }
729
730 // 按出点顺序显示前 count 个点 (仿真动画「点云逐步生成」)。
731 // 仅改写 drawRange, 不重建缓冲区, 可安全地每帧调用; count 传 Infinity 即完整显示。
732 function revealPointCloud(count) {
733   if (!state.pointCloud) return
734   const geo = state.pointCloud.geometry
735   const total = geo.getAttribute('position').count
736   geo.setDrawRange(0, Math.max(0, Math.min(total, Math.round(count))))
737 }
738
739 // 当前点云点数 (无点云时为 0)
740 function getPointCloudCount() {
741   return state.pointCloud ? state.pointCloud.geometry.getAttribute('position').count : 0
742 }

```

```

743 function computeColors(points, intensity, options) {
744     if (options.colorMode === 'height') return heightColors(points)
745     if (options.colorMode === 'intensity' && intensity && intensity.length) {
746         return intensityColors(intensity)
747     }
748     return null
749 }
750
751 function heightColors(points) {
752     let min = Infinity, max = -Infinity
753     for (const p of points) { if (p[2] < min) min = p[2]; if (p[2] > max) max = p[2] }
754     const span = max - min || 1
755     const colors = new Float32Array(points.length * 3)
756     for (let i = 0; i < points.length; i++) {
757         const c = heightColor((points[i][2] - min) / span)
758         colors[i * 3] = c[0]; colors[i * 3 + 1] = c[1]; colors[i * 3 + 2] = c[2]
759     }
760     return colors
761 }
762
763 function intensityColors(intensity) {
764     let min = Infinity, max = -Infinity
765     for (const v of intensity) { if (v < min) min = v; if (v > max) max = v }
766     const span = max - min || 1
767     const colors = new Float32Array(intensity.length * 3)
768     for (let i = 0; i < intensity.length; i++) {
769         const c = heightColor((intensity[i] - min) / span)
770         colors[i * 3] = c[0]; colors[i * 3 + 1] = c[1]; colors[i * 3 + 2] = c[2]
771     }
772     return colors
773 }
774
775 // ----- 渲染选项 -----
776
777 function setAxesVisible(visible) {
778     if (state.axesGroup) state.axesGroup.visible = visible
779 }
780
781 function setWaypointColor(color) {
782     state.waypointColor = color
783     state.waypointSpheres.forEach((s) => {
784         if (s.index !== state.selectedIndex) {
785             s.mesh.material.color.set(color)
786         }
787     })
788 }
789
790 function setTrajectoryColor(color) {
791     state.trajectoryColor = color
792     if (state.waypointLine) {
793         state.waypointLine.material.color.set(color)
794     }
795 }
796

```

```

797
798 function setArrowColor(color) {
799     state.arrowColor = color
800     state.waypointArrows.forEach((a) => a.setColor(color))
801 }
802
803 function setArrowsVisible(visible) {
804     state.waypointArrows.forEach((a) => { a.visible = visible })
805 }
806
807 return {
808     init, dispose,
809     loadModel, removeModel, clearModels,
810     setModelVisible, setModelBbox,
811     setPointCloud, updatePointCloud, clearPointCloud,
812     revealPointCloud, getPointCloudCount,
813     setWaypointCallbacks, renderWaypoints,
814     setPickMode,
815     onFrame, offFrame,
816     getSceneMaxZ,
817     get renderer() { return state.renderer },
818     get scene() { return state.scene },
819     get camera() { return state.camera },
820     get controls() { return state.controls },
821     get animGroup() { return state.animGroup },
822     setAxesVisible,
823     setWaypointColor,
824     setTrajectoryColor,
825     setArrowColor,
826     setArrowsVisible,
827 }
828 }

```

Listing 11: frontend/src/composables/useThreeScene.js

7.2.2. 航点交互系统

以下为 useWaypoints.js 的实现，作为面向组件的航点交互门面：

```

1 import { useThreeScene } from './useThreeScene'
2
3 // 航点交互门面：面向组件的高层 API。
4 // 底层航点渲染与交互（点击添加、拖拽、选中删除）由 useThreeScene 的场景管理器实现。
5 export function useWaypoints() {
6     const three = useThreeScene()
7     return {
8         // 注册交互回调：{onAdd, onMove, onRemove}
9         setCallbacks: (cb) => three.setWaypointCallbacks(cb),
10        // 以航点数组重建三维表示（球体 + 连线）
11        renderWaypoints: (points) => three.renderWaypoints(points),
12    }
13 }

```

Listing 12: frontend/src/composables/useWaypoints.js

8. 后端实现详解

8.1. FastAPI 应用架构

8.1.1. 主应用配置

FastAPI 应用入口 `main.py` 完成 CORS 中间件、静态文件目录与路由的装配：

```
1  """FastAPI应用入口：装配CORS、静态文件与路由。"""
2  from fastapi import FastAPI
3  from fastapi.middleware.cors import CORSMiddleware
4  from fastapi.staticfiles import StaticFiles
5
6  from app.api.routes import router as api_router
7  from app.api.websocket import router as ws_router
8  from app.config import CORS_ORIGINS, STATIC_DIR, ensure_dirs
9
10 app = FastAPI(title="FeHALSBackend", version="0.1.0")
11
12 app.add_middleware(
13     CORSMiddleware,
14     allow_origins=CORS_ORIGINS,
15     allow_credentials=True,
16     allow_methods=["*"],
17     allow_headers=["*"],
18 )
19
20
21 @app.on_event("startup")
22 async def _startup() -> None:
23     ensure_dirs()
24
25
26 # 静态文件（模型 / 航迹 / 配置 / 结果）
27 app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="static")
28
29 app.include_router(api_router)
30 app.include_router(ws_router)
31
32
33 @app.get("/")
34 async def root():
35     return {"service": "FeHALSBackend", "docs": "/docs"}
```

Listing 13: backend/app/main.py

8.1.2. 配置管理系统

全局配置 `config.py` 管理服务地址、静态目录与 HELIOS++ 路径，所有项均支持通过环境变量覆盖：

```
1  """全局配置：服务地址、静态目录、HELIOS++可执行文件与资源路径。
2
3  所有可调项均支持通过环境变量覆盖，便于在不同部署环境间切换。
4  """
5  import os
```

```

6 from pathlib import Path
7
8 # 服务监听地址与端口
9 HOST = os.getenv("FEHALS_HOST", "0.0.0.0")
10 PORT = int(os.getenv("FEHALS_PORT", "8000"))
11
12 # 后端静态文件根目录（模型 / 航迹 / 配置 / 结果）
13 BASE_DIR = Path(__file__).resolve().parent
14 STATIC_DIR = BASE_DIR / "static"
15 MODELS_DIR = STATIC_DIR / "models"
16 TRAJECTORIES_DIR = STATIC_DIR / "trajectories"
17 CONFIGS_DIR = STATIC_DIR / "configs"
18 RESULTS_DIR = STATIC_DIR / "results"
19
20 # HELIOS++ 可执行文件路径（helio++ v2.x）
21 HELIOS_PATH = os.getenv("HELIOS_PATH", "helio++")
22
23 # HELIOS++ 源仓库根目录（含 data/sceneparts、data/scenes 等演示资源）
24 _HELIOS_REPO = Path(os.getenv("HELIOS_REPO", "/home/azusa/file/project/3rd/helios"))
25
26 # HELIOS++ --assets 搜索路径：
27 # - 仓库根目录：解析 survey 中的 data/sceneparts、data/scenes 等演示资源
28 # - pyhelios 数据目录：解析 data/platforms.xml、data/scanners_*.xml 平台/扫描器目录
29 _DEFAULT_ASSETS = [
30     str(_HELIOS_REPO),
31     str(_HELIOS_REPO / "python" / "pyhelios"),
32 ]
33 HELIOS_ASSETS = [
34     p for p in os.getenv("HELIOS_ASSETS", os.pathsep.join(_DEFAULT_ASSETS)).split(os.pathsep) if p
35 ]
36
37 # 仿真超时时间（秒）
38 SIMULATION_TIMEOUT = int(os.getenv("FEHALS_SIM_TIMEOUT", "60000"))
39
40 # CORS 允许来源（开发环境放开）
41 CORS_ORIGINS = [o for o in os.getenv("FEHALS_CORS_ORIGINS", "*").split(",") if o]
42
43
44 def ensure_dirs() -> None:
45     """确保静态子目录存在。"""
46     for d in (MODELS_DIR, TRAJECTORIES_DIR, CONFIGS_DIR, RESULTS_DIR):
47         d.mkdir(parents=True, exist_ok=True)

```

Listing 14: backend/app/config.py

8.2. API 路由实现

8.2.1. REST 路由

routes.py 实现全部 REST 端点，包含模型、航迹、配置、仿真、结果、环境诊断与缓存管理七类资源操作：

```

1 """REST API 路由。
2
3 模型、航迹、配置、仿真、结果、缓存六类资源。

```

```

4  """
5  import json
6  import numpy as np
7  import shutil
8  import time
9  import uuid
10 from pathlib import Path
11
12 from fastapi import APIRouter, File, HTTPException, UploadFile
13 from fastapi.responses import FileResponse
14
15 from app.config import CONFIGS_DIR, MODELS_DIR, RESULTS_DIR, TRAJECTORIES_DIR
16 from app.models.schemas import (
17     ConfigRequest,
18     CoverageRequest,
19     SimulationRunRequest,
20     TrajectoryRequest,
21 )
22 from app.services import (
23     config_generator,
24     env_service,
25     helios_service,
26     pointcloud_parser,
27     trajectory_generator,
28 )
29 from ..services.coverage import CoverageAnalyzer
30 router = APIRouter(prefix="/api")
31
32 # 模型注册表: model_id -> {filename, path, url, ext} (单进程内有效)
33 MODEL_REGISTRY: dict = {}
34
35 # 允许上传的模型格式 (其中仅 .obj 可参与 HELIOS++ 仿真)
36 ALLOWED_MODEL_EXTS = {".obj", ".gltf", ".glb", ".stl"}
37
38 _CHUNK = 1024 * 1024
39
40
41 # ----- 模型管理 -----
42
43 def _restore_model_registry() -> None:
44     """启动时从模型目录恢复注册表。
45
46     注册表存于内存，后端重启即丢失；模型文件本身持久化在 MODELS_DIR。
47     上传时落盘到{model_id}.json清单（记录原始文件名），此处连同无清单的
48     孤儿模型文件一并恢复注册，避免后端重启后已上传模型“消失”。
49     """
50     for p in sorted(MODELS_DIR.iterdir()):
51         ext = p.suffix.lower()
52         if ext not in ALLOWED_MODEL_EXTS:
53             continue
54         model_id = p.stem
55         if model_id in MODEL_REGISTRY:
56             continue
57         manifest = MODELS_DIR / f"{model_id}.json"

```

```

58     filename = p.name
59     if manifest.exists():
60         try:
61             filename = json.loads(manifest.read_text(encoding="utf-8")).get("filename", p.name)
62         except (OSError, ValueError):
63             pass
64     MODEL_REGISTRY[model_id] = {
65         "filename": filename,
66         "path": str(p),
67         "url": f"/static/models/{p.name}",
68         "ext": ext,
69         "up": config_generator.detect_up_axis(str(p)) if ext == ".obj" else "z",
70     }
71
72
73 _restore_model_registry()
74
75
76 @router.post("/models/upload")
77 async def upload_model(file: UploadFile = File(...)):
78     ext = Path(file.filename or "").suffix.lower()
79     if ext not in ALLOWED_MODEL_EXTS:
80         raise HTTPException(
81             400, f"不支持的模型格式: {ext}或未知 (支持{' '.join(sorted(ALLOWED_MODEL_EXTS))}) "
82         )
83     model_id = f"mod_{int(time.time()*1000)}_{uuid.uuid4().hex[:6]}"
84     dest = MODELS_DIR / f"{model_id}{ext}"
85     # 分块流式写入，避免大文件占用内存
86     with open(dest, "wb") as f:
87         while chunk := await file.read(_CHUNK):
88             f.write(chunk)
89
90     url = f"/static/models/{dest.name}"
91     # 推断 OBJ 模型的 up 轴 (Y-up 需在 HELIOS++ 与前端一致地旋转到 Z-up)
92     up = config_generator.detect_up_axis(str(dest)) if ext == ".obj" else "z"
93     MODEL_REGISTRY[model_id] = {
94         "filename": file.filename or dest.name,
95         "path": str(dest),
96         "url": url,
97         "ext": ext,
98         "up": up,
99     }
100     # 落盘清单（原始文件名），供后端重启后恢复注册表
101     (MODELS_DIR / f"{model_id}.json").write_text(
102         json.dumps({"filename": file.filename or dest.name, ensure_ascii=False},
103             encoding="utf-8",
104         )
105     )
106     return {"model_id": model_id, "filename": file.filename, "url": url, "up": up}
107
108 @router.get("/models")
109 async def list_models():
110     models = [
111         {"id": mid, "name": m["filename"], "url": m["url"], "up": m.get("up", "z")}

```

```

112         for mid, m in MODEL_REGISTRY.items()
113     ]
114     return {"models": models}
115
116
117 @router.delete("/models/{model_id}")
118 async def delete_model(model_id: str):
119     m = MODEL_REGISTRY.pop(model_id, None)
120     if m:
121         Path(m["path"]).unlink(missing_ok=True)
122         (MODELS_DIR / f"{model_id}.json").unlink(missing_ok=True)
123     return {"success": True}
124
125
126 # ----- 航迹管理 -----
127
128 @router.post("/trajectory/generate")
129 async def generate_trajectory(req: TrajectoryRequest):
130     if not req.waypoints:
131         raise HTTPException(400, "航点列表不能为空")
132     result = trajectory_generator.generate(req.waypoints, req.altitude)
133     return {
134         "file_id": result["file_id"],
135         "download_url": f"/api/trajectories/download/{result['file_id']}",
136         "point_count": result["point_count"],
137     }
138
139
140 @router.get("/trajectories")
141 async def list_trajectories():
142     items = []
143     for p in sorted(TRAJECTORIES_DIR.glob("*.trj")):
144         items.append(
145             {"id": p.stem, "created": p.stat().st_mtime, "point_count": None}
146         )
147     return {"trajectories": items}
148
149
150 @router.get("/trajectories/download/{file_id}")
151 async def download_trajectory(file_id: str):
152     p = TRAJECTORIES_DIR / f"{file_id}.trj"
153     if not p.exists():
154         raise HTTPException(404, "航迹文件不存在")
155     return FileResponse(str(p), filename=f"{file_id}.trj", media_type="text/plain")
156
157
158 # ----- 配置管理 -----
159
160 @router.post("/config/generate")
161 async def generate_config(req: ConfigRequest):
162     config_id = config_generator.store_config(req.model_dump())
163     return {
164         "config_id": config_id,
165         "config_path": f"/static/configs/{config_id}.json",

```

```

166     }
167
168
169 # ----- 仿真执行 -----
170
171 @router.post("/simulation/run")
172 async def run_simulation(req: SimulationRunRequest):
173     # 1. 校验航迹
174     traj_path = TRAJECTORIES_DIR / f"{req.trajectory_id}.trj"
175     if not traj_path.exists():
176         raise HTTPException(404, f"航迹不存在: {req.trajectory_id}")
177
178     # 2. 校验配置
179     try:
180         params = config_generator.get_config(req.config_id)
181     except KeyError:
182         raise HTTPException(404, f"配置不存在: {req.config_id}")
183
184     # 3. 校验模型 (可选)
185     model_entries: list[tuple[str, str]] = []
186     if req.scene_model_ids:
187         for mid in req.scene_model_ids:
188             m = MODEL_REGISTRY.get(mid)
189             if not m:
190                 raise HTTPException(404, f"模型不存在: {mid}")
191             if m["ext"] != ".obj":
192                 raise HTTPException(400, f"模型「{m['filename']}」不是OBJ格式 (仅OBJ可参与HELIOS++
193                                     仿真)")
194             model_entries.append((m["path"], m.get("up", "z")))
195
196     # 4. 生成 scene + survey XML
197     scene_xml, scene_id = config_generator.generate_scene_xml(model_entries if model_entries else None)
198     survey_xml = config_generator.generate_survey_xml(
199         scene_xml_path=str(scene_xml),
200         scene_id=scene_id,
201         traj_path=str(traj_path),
202         params=params,
203     )
204
205     # 5. 启动仿真
206     output_dir = RESULTS_DIR / f"sim_{int(time.time()*1000)}_{uuid.uuid4().hex[:6]}"
207     output_dir.mkdir(parents=True, exist_ok=True)
208     output_format = params.get("output_format", "LAS")
209
210     task = helios_service.run_simulation(str(survey_xml), str(output_dir), output_format)
211     return {"task_id": task.task_id}
212
213 @router.get("/simulation/status/{task_id}")
214 async def simulation_status(task_id: str):
215     task = helios_service.get_task(task_id)
216     if task is None:
217         raise HTTPException(404, f"任务不存在: {task_id}")
218     return {

```

```

219         "task_id": task.task_id,
220         "status": task.status,
221         "progress": task.progress,
222         "message": task.message,
223         "result_file": task.result_file,
224     }
225
226
227 @router.get("/simulation/logs/{task_id}")
228 async def simulation_logs(task_id: str):
229     task = helios_service.get_task(task_id)
230     if task is None:
231         raise HTTPException(404, f"任务不存在: {task_id}")
232     return {"logs": task.logs}
233
234
235 @router.post("/simulation/cancel/{task_id}")
236 async def cancel_simulation(task_id: str):
237     ok = await helios_service.cancel(task_id)
238     return {"success": ok}
239
240
241 # ----- 结果管理 -----
242
243 @router.get("/results/{task_id}")
244 async def get_result(task_id: str):
245     task = helios_service.get_task(task_id)
246     if task is None:
247         raise HTTPException(404, f"任务不存在: {task_id}")
248     if task.status != "completed" or not task.result_file:
249         raise HTTPException(409, f"任务尚未完成或未产生结果 (当前状态: {task.status}) ")
250
251     parsed = pointcloud_parser.parse(task.result_file)
252     return {
253         "file_path": parsed["file_path"],
254         "point_count": parsed["point_count"],
255         "bounds": parsed["bounds"],
256         "stats": parsed["stats"],
257         "points": parsed["points"],
258         "intensity": parsed["intensity"],
259     }
260
261
262 @router.get("/results/{task_id}/download")
263 async def download_result(task_id: str, format: str = "las"):
264     task = helios_service.get_task(task_id)
265     if task is None:
266         raise HTTPException(404, f"任务不存在: {task_id}")
267     if not task.result_file:
268         raise HTTPException(409, f"任务尚未产生结果 (当前状态: {task.status}) ")
269     p = Path(task.result_file)
270     return FileResponse(str(p), filename=p.name, media_type="application/octet-stream")
271
272 # ----- 环境诊断 -----

```

```

273
274 @router.get("/env/diagnose")
275 async def diagnose_env():
276     """检测HELIOS++运行环境：可执行文件、资源目录完整性、静态工作目录。"""
277     return await env_service.diagnose_all()
278
279 # ----- 缓存管理 -----
280
281 CACHE_DIRS = {
282     "models": MODELS_DIR,
283     "trajectories": TRAJECTORIES_DIR,
284     "configs": CONFIGS_DIR,
285     "results": RESULTS_DIR,
286 }
287
288 # 缓存目录名 → 中文标签
289 CACHE_LABELS = {
290     "models": "模型",
291     "trajectories": "航迹",
292     "configs": "配置",
293     "results": "结果",
294 }
295
296
297 @router.get("/cache")
298 async def cache_stats():
299     """返回各缓存目录的文件数及总大小（字节）。"""
300     stats = {}
301     for key, d in CACHE_DIRS.items():
302         if not d.exists():
303             stats[key] = {"label": CACHE_LABELS.get(key, key), "count": 0, "size": 0}
304             continue
305         count = 0
306         size = 0
307         for f in d.iterdir():
308             if f.is_file() and f.name != ".gitkeep":
309                 count += 1
310                 size += f.stat().st_size
311         stats[key] = {"label": CACHE_LABELS.get(key, key), "count": count, "size": size}
312     return {"cache": stats}
313
314
315 @router.delete("/cache/{cache_type}")
316 async def clear_cache(cache_type: str):
317     """清空指定缓存目录（保留.gitkeep）。"""
318     d = CACHE_DIRS.get(cache_type)
319     if d is None:
320         raise HTTPException(404, f"未知缓存类型：{cache_type}（可选：{'、'}.join(CACHE_DIRS.keys())）")
321     if not d.exists():
322         return {"success": True, "type": cache_type, "removed": 0}
323     removed = 0
324     for f in d.iterdir():
325         if f.is_file() and f.name != ".gitkeep":
326             f.unlink()

```

```

327         removed += 1
328     return {"success": True, "type": cache_type, "removed": removed}
329 # ===== 点云覆盖度分析 =====
330
331 @router.post("/coverage/analyze")
332 async def analyze_coverage(request: CoverageRequest):
333     """分析点云覆盖度：投影到XY平面网格，返回密度矩阵与统计指标。"""
334     try:
335         points = np.array(request.points)
336         analyzer = CoverageAnalyzer(grid_size=request.grid_size)
337         return analyzer.analyze(points)
338     except Exception as e:
339         raise HTTPException(status_code=500, detail=str(e))

```

Listing 15: backend/app/api/routes.py

8.3. 业务服务层

8.3.1. 航迹生成服务

trajectory_generator.py 将用户航点列表转换为 HELIOS++ 原生.trj 航迹文件，包含偏航角计算、速度驱动时间步长与转角瞬时转向逻辑：

```

1  """生成HELIOS++兼容的航迹文件(.trj)。
2
3  HELIOS++原生航迹为CSV格式，列顺序为：
4  t,roll,pitch,yaw,x,y,z
5  (参见helios-demo/data/trajectories/flyandrotate.trj)
6  surveyXML中通过tIndex/xIndex/yIndex/zIndex/rollIndex/pitchIndex/yawIndex映射列。
7
8  偏航角(yaw)根据航向计算：
9  使用ARINC705规范，yaw=0表示+y方向(北)，顺时针增加
10 yaw=90表示+x方向(东)，yaw=-90(或270)表示-x方向(西)
11 若不设置正确的yaw，扫描器将沿错误方向扫描(沿轨而非正交)，导致点云形状异常
12 """
13 import math
14 import time
15 from typing import List
16
17 from app.config import TRAJECTORIES_DIR
18
19
20 def _compute_yaw(x1: float, y1: float, x2: float, y2: float) -> float:
21     """计算从点(x1,y1)到点(x2,y2)的偏航角(度)。
22
23     atan2(dx,dy): +y方向为0°, +x方向为90°, 返回[-180,180]。
24     """
25     dx = x2 - x1
26     dy = y2 - y1
27     if abs(dx) < 1e-9 and abs(dy) < 1e-9:
28         return 0.0
29     return math.degrees(math.atan2(dx, dy))
30
31
32 def generate(waypoints: List[List[float]], altitude: float = 100.0) -> dict:

```

```

33     """将航点列表写入原生.trj文件。
34
35     飞行高度为恒定值：所有航点的高程统一为altitude，
36     仅用航点的x、y定义水平飞行路径（ALS常规做法）。
37     偏航角由相邻航点间的方向向量实时计算，确保扫描器始终正交于航线。
38
39     每段航线的两端点yaw相同（指向该段方向），
40     方向变化处通过同一时间戳的重复行实现瞬时转向（同flyandrotate.trj做法）。
41     返回dict：{file_id,path,point_count}
42     """
43     file_id = f"traj_{int(time.time()*1000)}"
44     path = TRAJECTORIES_DIR / f"{file_id}.trj"
45
46     lines = [
47         "#TIME_COLUMN:0",
48         '#HEADER:"t","roll","pitch","yaw","x","y","z"',
49     ]
50
51     # 目标飞行速度（m/s），用于计算航段间的时间步长
52     # 典型 UAV 巡航速度 5~15 m/s，取 10 m/s 确保扫描线间距合理
53     TARGET_SPEED = 10.0
54
55     n = len(waypoints)
56     if n == 1:
57         # 单个航点，yaw 无意义，设为 0
58         x, y = float(waypoints[0][0]), float(waypoints[0][1])
59         lines.append(f"0,0,0,0.00,{x:.6f},{y:.6f},{altitude:.6f}")
60     else:
61         # 预先计算每段的时间和累积时间
62         cum_t = 0.0
63         seg_times = [0.0] # seg_times[i] = 航点 i 的时间
64         for i in range(n - 1):
65             dx = float(waypoints[i + 1][0]) - float(waypoints[i][0])
66             dy = float(waypoints[i + 1][1]) - float(waypoints[i][1])
67             dist = math.sqrt(dx * dx + dy * dy)
68             dt = max(1.0, dist / TARGET_SPEED)
69             cum_t += dt
70             seg_times.append(cum_t)
71
72         for i in range(n - 1):
73             x, y = float(waypoints[i][0]), float(waypoints[i][1])
74             next_x, next_y = float(waypoints[i + 1][0]), float(waypoints[i + 1][1])
75             yaw = _compute_yaw(x, y, next_x, next_y)
76             t = seg_times[i]
77
78             if i > 0:
79                 # 中间航点：先写上一段方向的结束行，再写本段方向的起始行
80                 # 两行在同一时间戳，实现瞬时转向
81                 prev_x, prev_y = float(waypoints[i - 1][0]), float(waypoints[i - 1][1])
82                 incoming_yaw = _compute_yaw(prev_x, prev_y, x, y)
83                 lines.append(f"{t:.2f},0,0,{incoming_yaw:.2f},{x:.6f},{y:.6f},{altitude:.6f}")
84
85                 lines.append(f"{t:.2f},0,0,{yaw:.2f},{x:.6f},{y:.6f},{altitude:.6f}")
86

```

```

87     # 最后一个航点：沿用上一段的方向
88     x, y = float(waypoints[-1][0]), float(waypoints[-1][1])
89     prev_x, prev_y = float(waypoints[-2][0]), float(waypoints[-2][1])
90     yaw = _compute_yaw(prev_x, prev_y, x, y)
91     lines.append(f"{seg_times[-1]:.2f},0,0,{yaw:.2f},{x:.6f},{y:.6f},{altitude:.6f}")
92
93     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
94     return {"file_id": file_id, "path": str(path), "point_count": n}

```

Listing 16: backend/app/services/trajectory_generator.py

8.3.2. 点云处理服务

pointcloud_parser.py 解析 LAS/LAZ 与 XYZ 格式的点云，计算全量统计（均值/标准差/中位数/P5/P95/直方图）并提供百万级显示抽样：

```

1  """点云解析：读取HELIOS++输出的LAS/LAZ/XYZ文件，提取坐标与统计信息。
2
3  laspy用于LAS/LAZ，纯文本解析用于XYZ。返回可供前端渲染的降采样点集。
4  """
5  from pathlib import Path
6  from typing import Optional
7
8  import numpy as np
9
10 # 前端渲染点上限（超过则降采样）
11 MAX_RENDER_POINTS = 1_000_000
12
13 # 高度分布直方图分箱数
14 HISTOGRAM_BINS = 40
15
16
17 def parse(file_path: str, max_points: int = MAX_RENDER_POINTS) -> dict:
18     """解析点云文件，返回{file_path, point_count, bounds, stats, points, intensity}。
19
20     points为降采样后的[[x, y, z], ...]列表；intensity为对应强度（若无则None）；
21     stats为全量点的特征统计（高度统计、强度统计、高度分布直方图，不受降采样影响）。
22     """
23     p = Path(file_path)
24     if not p.exists():
25         raise FileNotFoundError(f"点云文件不存在：{file_path}")
26
27     ext = p.suffix.lower()
28     if ext in (".las", ".laz"):
29         return _parse_las(p, max_points)
30     if ext in (".xyz", ".txt"):
31         return _parse_xyz(p, max_points)
32     raise ValueError(f"不支持的点云格式：{ext}")
33
34
35 def _parse_las(p: Path, max_points: int) -> dict:
36     import laspy # 延迟导入，避免无laspy时启动失败
37
38     las = laspy.read(str(p))

```

```

39 xyz = np.vstack((las.x, las.y, las.z)).transpose()
40 intensity = np.asarray(las.intensity) if hasattr(las, "intensity") else None
41 return _build_result(str(p), xyz, intensity, max_points)
42
43
44 def _parse_xyz(p: Path, max_points: int) -> dict:
45     # HELIOS++ 默认 ASCII 点云输出列序 (见 DirectMeasurementWriteStrategy.h) :
46     #   x y z intensity echo_width returnNumber pulseReturnNumber fullwaveIndex ...
47     # 逐行流式读取; 分块转为 ndarray, 避免千万级行的 Python 列表整体驻留内存
48     _CHUNK_ROWS = 500_000
49     blocks: list = []
50     inten_blocks: list = []
51     buf: list = []
52     ibuf: list = []
53     has_inten = True # 任一有效行缺第 4 列即整体丢弃强度 (与旧实现语义一致)
54     with open(p, "r", encoding="utf-8", errors="replace") as f:
55         for line in f:
56             parts = line.split()
57             if len(parts) < 3:
58                 continue
59             try:
60                 buf.append((float(parts[0]), float(parts[1]), float(parts[2])))
61                 if len(parts) > 3 and has_inten:
62                     ibuf.append(float(parts[3]))
63             else:
64                 has_inten = False
65             except ValueError:
66                 continue
67             if len(buf) >= _CHUNK_ROWS:
68                 blocks.append(np.asarray(buf, dtype=np.float64))
69                 inten_blocks.append(np.asarray(ibuf, dtype=np.float64))
70                 buf, ibuf = [], []
71
72     if buf:
73         blocks.append(np.asarray(buf, dtype=np.float64))
74         inten_blocks.append(np.asarray(ibuf, dtype=np.float64))
75     if not blocks:
76         return _build_result(str(p), np.empty((0, 3)), None, max_points)
77     xyz = blocks[0] if len(blocks) == 1 else np.vstack(blocks)
78     intensity = np.concatenate(inten_blocks) if has_inten and len(inten_blocks) else None
79     return _build_result(str(p), xyz, intensity, max_points)
80
81 def _build_result(path: str, xyz: np.ndarray, intensity: Optional[np.ndarray], max_points: int) -> dict:
82     :
83     n = xyz.shape[0]
84     bounds = (
85         [float(xyz[:, 0].min()), float(xyz[:, 1].min()), float(xyz[:, 2].min()),
86          float(xyz[:, 0].max()), float(xyz[:, 1].max()), float(xyz[:, 2].max())]
87         if n else [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
88     )
89     # 特征统计基于全量点 (降采样仅影响渲染点集)
90     point_stats = stats(xyz, intensity)
91     # 降采样

```

```

92     if n > max_points:
93         idx = np.linspace(0, n - 1, max_points).astype(int)
94         xyz = xyz[idx]
95         intensity = intensity[idx] if intensity is not None else None
96
97     result = {
98         "file_path": path,
99         "point_count": n,
100        "bounds": bounds,
101        "stats": point_stats,
102        "points": xyz.astype(float).round(6).tolist(),
103        "intensity": intensity.astype(float).round(4).tolist() if intensity is not None else None,
104    }
105    return result
106
107
108 def stats(xyz: np.ndarray, intensity: Optional[np.ndarray] = None) -> dict:
109     """点云特征统计：点数、高度统计（均值/标准差/中位数/P5/P95/范围）、
110     强度统计（有强度数据时）与高度分布直方图。均基于传入的全量点集。
111     """
112     n = xyz.shape[0]
113     if n == 0:
114         return {"count": 0}
115     z = xyz[:, 2]
116     result = {
117         "count": int(n),
118         "mean_z": float(z.mean()),
119         "std_z": float(z.std()),
120         "min_z": float(z.min()),
121         "max_z": float(z.max()),
122         "median_z": float(np.percentile(z, 50)),
123         "p05_z": float(np.percentile(z, 5)),
124         "p95_z": float(np.percentile(z, 95)),
125         "z_histogram": _z_histogram(z),
126     }
127     if intensity is not None and intensity.shape[0] == n:
128         result["intensity"] = {
129             "mean": float(intensity.mean()),
130             "std": float(intensity.std()),
131             "min": float(intensity.min()),
132             "max": float(intensity.max()),
133         }
134     return result
135
136
137 def _z_histogram(z: np.ndarray, bins: int = HISTOGRAM_BINS) -> dict:
138     """高度分布直方图（等宽分箱），供前端渲染分布图。"""
139     z_min, z_max = float(z.min()), float(z.max())
140     if z_min == z_max:
141         return {"min": z_min, "max": z_max, "bin_size": 0.0, "bins": [int(z.shape[0])]}
142     counts, _ = np.histogram(z, bins=bins, range=(z_min, z_max))
143     return {
144         "min": z_min,
145         "max": z_max,

```

```
146     "bin_size": (z_max - z_min) / bins,  
147     "bins": counts.astype(int).tolist(),  
148 }
```

Listing 17: backend/app/services/pointcloud_parser.py

参考文献

- [1] Khronos Group. WebGL specification[EB/OL]. 2011. <https://www.khronos.org/registry/webgl/specs/latest/>.
- [2] COLVIN S. Pydantic: data validation using python type hints[EB/OL]. 2019. <https://docs.pydantic.dev/>.
- [3] FIELDING R T. Architectural styles and the design of network-based software architectures[D/OL]. University of California, Irvine, 2000. <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [4] FIELDING R, NOTTINGHAM M, RESCHKE J. HTTP semantics: 9110[R/OL]. RFC Editor, 2022. <https://www.rfc-editor.org/info/rfc9110>. DOI: 10.17487/RFC9110.
- [5] THOMSON M, BENFIELD C. HTTP/2: 9113[R/OL]. RFC Editor, 2022. <https://www.rfc-editor.org/info/rfc9113>. DOI: 10.17487/RFC9113.
- [6] BISHOP M. HTTP/3: 9114[R/OL]. RFC Editor, 2022. <https://www.rfc-editor.org/info/rfc9114>. DOI: 10.17487/RFC9114.
- [7] OpenJS Foundation. Node.js[EB/OL]. 2009. <https://nodejs.org/>.
- [8] MONTAIGU T. laspy: python library for lidar LAS/LAZ IO[EB/OL]. 2017. <https://laspy.readthedocs.io/>.

A. 部署与配置管理

A.1. 环境变量配置

表 9: 环境变量配置说明

环境变量	说明	默认值
FEHALS_HOST	服务器监听地址	0.0.0.0
FEHALS_PORT	服务器端口	8000
HELIOS_PATH	HELIOS++ 可执行文件路径	/usr/local/bin/helios++
HELIOS_REPO	HELIOS++ 仓库路径	/opt/helios++/repo
HELIOS_ASSETS	HELIOS++ 资源目录	/opt/helios++/assets
FEHALS_SIM_TIMEOUT	仿真超时时间（秒）	300
FEHALS_CORS_ORIGINS	允许的跨域源	http://localhost:3000
FEHALS_UPLOAD_DIR	文件上传目录	uploads
FEHALS_CACHE_DIR	缓存目录	cache

A.2. Docker 部署方案

```
1 # Dockerfile
2 FROM python:3.9-slim
3
4 WORKDIR /app
5
6 # 安装系统依赖
7 RUN apt-get update && apt-get install -y \
8     gcc \
9     g++ \
10    git \
11    wget \
12    && rm -rf /var/lib/apt/lists/*
13
14 # 安装HELIOS++
15 RUN wget https://github.com/3dgeo-heidelberg/helios_plusplus/releases/download/v1.0.0/helios++_linux_v1
16     .0.0.tar.gz && \
17     tar -xzf helios++_linux_v1.0.0.tar.gz && \
18     mv helios++ /usr/local/bin/
19
20 # 复制requirements文件并安装Python依赖
21 COPY requirements.txt .
22 RUN pip install --no-cache-dir -r requirements.txt
23
24 # 复制应用代码
25 COPY . .
26
27 # 创建必要的目录
28 RUN mkdir -p uploads cache static logs
29
30 # 设置环境变量
31 ENV FEHALS_HOST=0.0.0.0
32 ENV FEHALS_PORT=8000
```

```

32 ENV HELIOS_PATH=/usr/local/bin/helios++
33 ENV HELIOS_REPO=/app/helios++/repo
34 ENV HELIOS_ASSETS=/app/helios++/assets
35
36 # 暴露端口
37 EXPOSE 8000
38
39 # 启动命令
40 CMD ["python", "run.py"]

```

Listing 18: Dockerfile - 容器化部署

A.3. 服务监控与日志

```

1 # 使用Prometheus + Grafana 监控
2 # prometheus.yml
3 global:
4     scrape_interval: 15s
5
6 scrape_configs:
7     - job_name: 'fehals'
8       static_configs:
9         - targets: ['fehals-backend:8000']
10      metrics_path: '/metrics'
11      scrape_interval: 15s

```

Listing 19: 监控配置示例

B. 开发规范与最佳实践

B.1. 代码规范

- **Python 代码**：遵循 PEP 8 规范，使用 black 格式化，flake8 检查
- **Vue 代码**：使用 ESLint + Prettier，采用 Composition API
- **Git 提交规范**：使用 Angular 提交格式 (type(scope): message)
- **分支管理**：master (主分支)、develop (开发分支)、feature/* (功能分支)

B.2. API 设计原则

- **RESTful**：遵循 REST 设计规范，使用标准 HTTP 方法
- **状态码**：使用标准 HTTP 状态码，如 200、201、400、404、500 等
- **错误处理**：统一错误格式，包含错误代码和描述信息
- **文档化**：所有 API 接口必须有 Swagger/OpenAPI 文档

C. 故障排查指南

表 10: 常见问题及解决方案

问题现象	解决方案
HELIOS++ 无法启动	检查 HELIOS_PATH 环境变量，确认可执行文件存在
模型文件上传失败	检查文件格式和大小，确认在允许范围内
仿真任务卡住	检查 HELIOS++ 资源目录，查看任务日志
WebSocket 连接断开	检查 CORS 配置，确认代理设置正确
点云渲染性能差	使用降采样，减少同时渲染的点数